Programming in Modern C++

Module M31: Virtual Function Table

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional

Weekly Recap

- Understood type casting – implicit as well as explicit – for built-in types, unrelated types, and classes on a hierarchy
- Understood the notions of upcast and downcast
- Understood Static and Dynamic Binding for Polymorphic type
- Understood `virtual` destructors, Pure Virtual Functions, and Abstract Base Class
- Designed the solution for a staff salary processing problem using iterative refinement – starting with a simple C solution and repeatedly refining finally to an easy, efficient, and extensible C++ solution based on flexible polymorphic hierarchy

Module Objectives

- Introduce a new C solution with function pointers
- Understand Virtual Function Table for dynamic binding (polymorphic dispatch)

Module Outline

Staff Salary Processing: New C Solution

Staff Salary Processing: Problem Statement: RECAP (Module 29)

- An organization needs to develop a salary processing application for its staff
- At present it has an engineering division only where Engineers and Managers work. Every Engineer reports to some Manager. Every Manager can also work like an Engineer
- The logic for processing salary for Engineers and Managers are different as they have different salary heads
- In future, it may add Directors to the team. Then every Manager will report to some Director. Every Director could also work like a Manager
- The logic for processing salary for Directors will also be distinct
- Further, in future it may open other divisions, like Sales division, and expand the workforce
- **Make a suitable extensible design**

C Solution: Function Pointers
Engineer + Manager + Director: RECAP (Module 29)

- How to represent Engineers, Managers, and Directors?
  - Collection of `structs`
- How to initialize objects?
  - Initialization functions
- How to have a collection of mixed objects?
  - Array of `union`
- How to model variations in salary processing algorithms?
  - `struct`-specific functions
- How to invoke the correct algorithm for a correct employee type?
  - Function switch
  - **Function pointers**

C Solution: Function Pointers: Engineer + Manager + Director

- In Module 29, we have developed a flat C Solution using *function switch*
- In Module 30, we refined the C Solution to develop two types of C++ Solution using
  - Non-polymorphic hierarchy - employing *function switch*
  - Polymorphic hierarchy - eomploying *virtual function*
- In Module 29, we had mentioned that in the flat C Solution it is not easy to use function pointers as the processing functions `void ProcessSalaryEngineer(Engineer *)`, `void ProcessSalaryManager(Manager *)`, and `void ProcessSalaryDirector(Director *)` all have different types of arguments and therefore a common function pointer type cannot be defined
- We can work around this by:
  - Passing the staff object as `void *`, instead of `Engineer *`, `Manager *`, or `Director *`
  - Cast it to respective object type in the respective function. That is, cast to `Engineer *` in `ProcessSalaryEngineer(Engineer *)` and so on
  - We can then use a function pointer type `void (*)(void *)`
- We illustrate in the Solution

C Solution: Function Pointers: Engineer + Manager + Director

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
typedef enum E_TYPE { Er, Mgr, Dir } E_TYPE; // Staff tag type
typedef void (*psFuncPtr)(void *);           // Processing func. ptr. type, passing the object by void *
typedef struct Engineer { char *name_; } Engineer; // Engineer Type
Engineer *InitEngineer(const char *name) { Engineer *e = (Engineer *)malloc(sizeof(Engineer));
    e->name_ = strdup(name); return e;
}
void ProcessSalaryEngineer(void *v) { Engineer *e = (Engineer *)v; // Cast explicitly to the staff object
    printf("%s: Process Salary for Engineer\n", e->name_);
}
typedef struct Manager { char *name_; Engineer *reports_[10]; } Manager; // Manager Type
Manager *InitManager(const char *name) { Manager *m = (Manager *)malloc(sizeof(Manager));
    m->name_ = strdup(name); return m;
}
void ProcessSalaryManager(void *v) { Manager *m = (Manager *)v; // Cast explicitly to the staff object
    printf("%s: Process Salary for Manager\n", m->name_);
}
typedef struct Director { char *name_; Manager *reports_[10]; } Director; // Director Type
Director *InitDirector(const char *name) { Director *d = (Director *)malloc(sizeof(Director));
    d->name_ = strdup(name); return d;
}
void ProcessSalaryDirector(void *v) { Director *d = (Director *)v; // Cast explicitly to the staff object
    printf("%s: Process Salary for Director\n", d->name_);
}
```

C Solution: Function Pointers: Engineer + Manager + Director


```
typedef struct Staff {
    E_TYPE type_; // Staff tag type
    void *p;      // Pointer to staff object
} Staff;          // Staff object wrapper
int main() {
    // Array of function pointers
    psFuncPtr psArray[] = { ProcessSalaryEngineer,  ProcessSalaryManager, ProcessSalaryDirector };

    // Array of staffs
    Staff staff[] = { { Er, InitEngineer("Rohit") }, { Mgr, InitEngineer("Kamala") },
                      { Mgr, InitEngineer("Rajib") }, { Er, InitEngineer("Kavita") },
                      { Er, InitEngineer("Shambhu") }, { Dir, InitEngineer("Ranjana") } };

    for (int i = 0; i < sizeof(staff) / sizeof(Staff); ++i)
        psArray[staff[i].type_] // Pick the right processing function for the tag - staff type
            (staff[i].p);       // Pass the pointer to the object - implicitly cast to void*
}
```

```
Rohit: Process Salary for Engineer
Kamala: Process Salary for Manager
Rajib: Process Salary for Manager
Kavita: Process Salary for Engineer
Shambhu: Process Salary for Engineer
Ranjana: Process Salary for Director
```

C Solution: Advantages and Disadvantages: RECAP (Module 26) Annotated for Function Pointers

- **Advantages**
  - Solution exists!
  - Code is well structured – has patterns
- **Disadvantages**
  - Employee data has scope for better organization
    - ▷ No encapsulation for data
    - ▷ Duplication of fields across types of employees – possible to mix up types for them (say, `char *` and `string`)
    - ▷ Employee objects are created and initialized dynamically through `Init...` functions. How to release the memory?
  - Types of objects are managed explicitly by `E_Type`:
    - ▷ Difficult to extend the design – addition of a new type needs to:
      - — Add new type code to `enum E_Type`
      - — Add a new pointer field in `struct Staff` for the new type
      - — Add a new case (`if-else` or `case`) based on the new type: **Removed using function pointer**
    - ▷ Error prone – developer has to decide to call the right processing function for every type (`ProcessSalaryManager` for `Mgr` etc.): **Removed using function pointer**
  - Unable to use Function Pointers as each processing function takes a parameter of different type - no common signature for dispatch
- **Recommendation**
  - Use classes for encapsulation on a hierarchy

Staff Salary Processing: C++ Solution

C++ Solution: Polymorphic Hierarchy: RECAP Engineer + Manager + Director: (Module 30)

- How to represent Engineers, Managers, and Directors?
  - Polymorphic `class` hierarchy
- How to initialize objects?
  - Constructor / Destructor
- How to have a collection of mixed objects?
  - `array` of base class pointers
- How to model variations in salary processing algorithms?
  - Member functions
- How to invoke the correct algorithm for a correct employee type?
  - Virtual Functions

C++ Solution: Polymorphic Hierarchy: RECAP Engineer + Manager + Director: (Module 30)


```
#include <iostream>
#include <string>
using namespace std;

class Engineer {
protected:
    string name_;
public:
    Engineer(const string& name) : name_(name) { }
    virtual ~Engineer() { }
    virtual void ProcessSalary() { cout << name_ << ": Process Salary for Engineer" << endl; }
};
class Manager : public Engineer {
    Engineer *reports_[10];
public:
    Manager(const string& name) : Engineer(name) { }
    void ProcessSalary() { cout << name_ << ": Process Salary for Manager" << endl; }
};
class Director : public Manager {
    Manager *reports_[10];
public:
    Director(const string& name) : Manager(name) { }
    void ProcessSalary() { cout << name_ << ": Process Salary for Director" << endl; }
};
```

C++ Solution: Polymorphic Hierarchy: RECAP Engineer + Manager + Director: (Module 30)


```
int main() {
    Engineer e1("Rohit"), e2("Kavita"), e3("Shambhu");
    Manager m1("Kamala"), m2("Rajib");
    Director d("Ranjana");
    Engineer *staff[] = { &e1, &m1, &m2, &e2, &e3, &d };

    for (int i = 0; i < sizeof(staff) / sizeof(Engineer*); ++i)
        staff[i]->ProcessSalary();
}
```

```
Rohit: Process Salary for Engineer
Kamala: Process Salary for Manager
Rajib: Process Salary for Manager
Kavita: Process Salary for Engineer
Shambhu: Process Salary for Engineer
Ranjana: Process Salary for Director
```

C and C++ Solutions: A Comparison

C and C++ Solutions: A Comparison

C Solution	C++ Solution

• How to represent Engineers, Managers, and Directors?
  ○ `structs`
• How to initialize objects?
  ○ Initialization functions
• How to have a collection of mixed objects?
  ○ `array` of `union` wrappers
• How to model variations in salary processing algorithms?
  ○ functions for `structs`
• How to invoke the correct algorithm for a correct employee type?
  ○ Function pointers
	• How to represent Engineers, Managers, and Directors?
	  ○ Polymorphic hierarchy
	• How to initialize objects?
	  ○ Ctor / Dtor
	• How to have a collection of mixed objects?
	  ○ `array` of base class pointers
	• How to model variations in salary processing algorithms?
	  ○ `class` member functions
	• How to invoke the correct algorithm for a correct employee type?
	  ○ Virtual Functions

C and C++ Solutions: A Comparison

C Solution (Function Pointer)

```
typedef enum E_TYPE { Er, Mgr, Dir } E_TYPE;
typedef void (*psFuncPtr)(void *);
typedef struct { E_TYPE type_; void *p; } Staff;
typedef struct { char *name_; } Engineer;
Engineer *InitEngineer(const char *name);
void ProcessSalaryEngineer(void *v);
typedef struct { char *name_; } Manager;
Manager *InitManager(const char *name);
void ProcessSalaryManager(void *v);
typedef struct { char *name_; } Director;
Director *InitDirector(const char *name);
void ProcessSalaryDirector(void *v);
int main() { psFuncPtr psArray[] = {
    ProcessSalaryEngineer,    // Function
    ProcessSalaryManager,     // pointer
    ProcessSalaryDirector };  // array
    Staff staff[] = {
    { Er, InitEngineer("Rohit") },
    { Mgr, InitEngineer("Kamala") },
    { Dir, InitEngineer("Ranjana") } };
    for (int i = 0; i <
        sizeof(staff)/sizeof(Staff); ++i)
        psArray[staff[i].type_](staff[i].p);
}
```

C++ Solution (Virtual Function)

```
class Engineer { protected: string name_;
public: Engineer(const string& name);
     virtual void ProcessSalary(); };
     virtual ~Engineer(); };
class Manager : public Engineer {
public: Manager(const string& name);
    void ProcessSalary(); };
class Director : public Manager {
public: Director(const string& name);
    void ProcessSalary(); };
int main() {
    // Function pointer array is subsumed in
    // virtual function tables of classes

    Engineer e1("Rohit");
    Manager m1("Kamala");
    Director d("Ranjana");
    Engineer *staff[] = { &e1, &m1, &d };
    for(int i = 0; i <
        sizeof(staff)/sizeof(Engineer*); ++i)
        staff[i]->ProcessSalary();
}
```

Virtual Function Pointer Table

How do virtual functions work?

- The C Solution with function pointers gives us the lead to implement virtual functions. Here
  - We have used an array of function pointers (`psFuncPtr psArray[]`) to keep the processing functions (`void ProcessSalaryEngineer(Engineer *)`, `void ProcessSalaryManager(Manager *)`, and `void ProcessSalaryDirector(Director *)`) indexed by the type tag (`enum E_TYPE { Er, Mgr, Dir }`)
  - In C++, every class is a separate type - so the tag can be removed if we bind this table (**Virtual Function Table** or VFT) with the class
  - Every class can have a VFT with its appropriate processing function pointer put there
  - By override, all these functions can have the same signature (`void ProcessSalary()`) and can be called through the same expression (`(Engineer *)->ProcessSalary()`)
- We now illustrate Virtual Function Table through simple examples to show how does it work for inherited, overridden and overloaded member functions

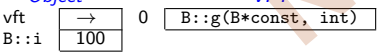
Virtual Function Pointer Table

Base Class

```
class B {
    int i;
public:
    B(int i_): i(i_) { }
        void f(int); // B::f(B*const, int)
virtual void g(int); // B::g(B*const, int)
};

B b(100);
B *p = &b;
```

b Object Layout



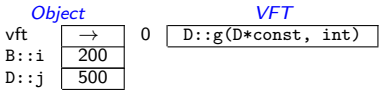
Source Expression	Compiled Expression
`b.f(15);`	`B::f(&b, 15);`
`p->f(25);`	`B::f(p, 25);`
`b.g(35);`	`B::g(&b, 35);`
`p->g(45);`	`p->vft[0](p, 45);`

Derived Class

```
class D: public B {
    int j;
public:
    D(int i_, int j_): B(i_), j(j_) { }
        void f(int); // D::f(D*const, int)
        void g(int); // D::g(D*const, int)
};

D d(200, 500);
B *p = &d;
```

d Object Layout



Source Expression	Compiled Expression
`d.f(15);`	`D::f(&d, 15);`
`p->f(25);`	`B::f(p, 25);`
`d.g(35);`	`D::g(&d, 35);`
`p->g(45);`	`p->vft[0](p, 45);`

Virtual Function Pointer Table

- Whenever a class defines a `virtual` function a hidden member variable is added to the class which points to an array of pointers to (`virtual`) functions called the **Virtual Function Table** (VFT)
- VFT pointers are used at run-time to invoke the appropriate function implementations, because at compile time it may not yet be known if the base function is to be called or a derived one implemented by a class that inherits from the base class
- VFT is class-specific – all instances of the class has the same VFT
- VFT carries the **Run-Time Type Information** (RTTI) of objects

Virtual Function Pointer Table

```
class A { public:
    virtual void f(int) { }
    virtual void g(double) { }
    int h(A *) { }
};
class B: public A { public:
    void f(int) { }
    virtual int h(B *) { }
};
class C: public B { public:
    void g(double) { }
    int h(B *) { }
};
A a; B b; C c;
A *pA; B *pB;
```

Source Expression	Compiled Expression
pA->f(2);	pA->vft[0](pA, 2);
pA->g(3.2);	pA->vft[1](pA, 3.2);
pA->h(&a);	A::h(pA, &a);
pA->h(&b);	A::h(pA, &b);
pB->f(2);	pB->vft[0](pB, 2);
pB->g(3.2);	pB->vft[1](pB, 3.2);
pB->h(&a);	pB->vft[2](pB, &a);
pB->h(&b);	pB->vft[2](pB, &b);

a **Object Layout**

Object vft → *VFT*

0	A::f(A*const, int)	Defined
1	A::g(A*const, double)	Defined

b **Object Layout**

Object vft → *VFT*

0	B::f(B*const, int)	Overridden
1	A::g(A*const, double)	Inherited
2	B::h(B*const, B*)	Overloaded

c **Object Layout**

Object vft → *VFT*

0	B::f(B*const, int)	Inherited
1	C::g(C*const, double)	Overridden
2	C::h(C*const, B*)	Overridden

Module Summary

- Leveraging an innovative solution to the Salary Processing Application in C using function pointers, we compare C and C++ solutions to the problem
- The new C solution with function pointers is used to explain the mechanism for dynamic binding (polymorphic dispatch) based on `virtual` function tables

Programming in Modern C++

Module M32: Type Casting & Cast Operators: Part 1

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional

Module Recap

- Leveraging an innovative solution to the Salary Processing Application in C using function pointers, we compare C and C++ solutions to the problem
- The new C solution with function pointers is used to explain the mechanism for dynamic binding (polymorphic dispatch) based on `virtual` function tables

Module Objectives

- Understand casting in C and C++
- Understand `const_cast` operator

Module Outline

Type Casting

Type Casting

- Why type casting?
  - Type casts are used to convert the type of an object, expression, function argument, or return value to that of another type
- (Silent) Implicit conversions
  - The standard C++ conversions and user-defined conversions
- Explicit conversions
  - Often the type needed for an expression that cannot be obtained through an implicit conversion. There may be more than one standard conversion that my create an ambiguous situation or there may be disallowed conversion. We need explicit conversion in such cases
- To perform a type cast, the compiler
  - Allocates temporary storage
  - Initializes temporary with value being cast

```
double f (int i,int j) { return (double) i / j; }

// compiler generates
double f (int i, int j) {
    double temp_i = i; // Explicit conversion by (double) in temporary
    double temp_j = j; // Implicit conversion in temporary to support mixed mode
    return temp_i / temp_j;
}
```

Casting: C-Style: RECAP (Module 26)

- Various type castings are possible between built-in types

```
int i = 3;
double d = 2.5;

double result = d / i; // i is cast to double and used
```

- Casting rules are defined between numerical types, between numercial types and pointers, and between pointers to different numerical types and `void`
- Casting can be **implicit** or **explicit**

```
int i = 3;
double d = 2.5, *p = &d;

d = i;          // implicit: int to double
i = d;          // implicit: warning: '=' : conversion from 'double' to 'int': possible loss of data

d = (double)i; // explicit: int to double
i = (int)d;    // explicit: double to int

i = p;          // error: '=' : cannot convert from 'double *' to 'int'
i = (int)p;    // explicit: double * to int
```

Casting: C-Style: RECAP (Module 26)

- (**Implicit**) Casting between *unrelated classes is not permitted*

```
class A { int i; };
class B { double d; };

A a;
B b;

A *p = &a;
B *q = &b;

a = b;     // error: binary '=' : no operator which takes a right-hand operand of type 'B'
a = (A)b;  // error: 'type cast' : cannot convert from 'B' to 'A'

b = a;     // error: binary '=' : no operator which takes a right-hand operand of type 'A'
b = (B)a;  // error: 'type cast' : cannot convert from 'A' to 'B'

p = q;     // error: '=' : cannot convert from 'B *' to 'A *'
q = p;     // error: '=' : cannot convert from 'A *' to 'B *'

p = (A*)&b; // explicit on pointer: type cast is okay for the compiler
q = (B*)&a; // explicit on pointer: type cast is okay for the compiler
```

Casting: C-Style: RECAP (Module 26)

- **Forced** Casting between *unrelated classes is dangerous*

```
class A { public: int i; };
class B { public: double d; };

A a;
B b;

a.i = 5;
b.d = 7.2;

A *p = &a;
B *q = &b;

cout << p->i << endl; // prints 5
cout << q->d << endl; // prints 7.2

p = (A*)&b; // Forced casting on pointer: Dangerous
q = (B*)&a; // Forced casting on pointer: Dangerous

cout << p->i << endl; // prints -858993459:    GARBAGE
cout << q->d << endl; // prints -9.25596e+061: GARBAGE
```

Casting on a Hierarchy: C-Style: RECAP (Module 26)

- Casting on a **hierarchy** is *permitted in a limited sense*

```
class A { };
class B : public A { };

A *pa = 0;
B *pb = 0;
void *pv = 0;

pa = pb; // UPCAST: Okay

pb = pa; // DOWNCAST: error: '=' : cannot convert from 'A *' to 'B *'

pv = pa; // Okay, but lose the type for A * to void *
pv = pb; // Okay, but lose the type for B * to void *

pa = pv; // error: '=' : cannot convert from 'void *' to 'A *'
pb = pv; // error: '=' : cannot convert from 'void *' to 'B *'
```

Casting on a Hierarchy: C-Style: RECAP (Module 26)

- **Up-Casting** is *safe*

```
class A { public: int dataA_; };
class B : public A { public: int dataB_; };

A a;
B b;

a.dataA_ = 2;
b.dataA_ = 3;
b.dataB_ = 5;

A *pa = &a;
B *pb = &b;

cout << pa->dataA_ << endl;                    // prints 2
cout << pb->dataA_ << " " << pb->dataB_ << endl; // prints 3 5

pa = &b;

cout << pa->dataA_ << endl;                    // prints 3
cout << pa->dataB_ << endl;                    // error: 'dataB_' : is not a member of 'A'
```

Cast Operators

Casting in C and C++

- Casting in C
  - Implicit cast
  - Explicit C-Style cast
  - Loses type information in several contexts
  - Lacks clarity of semantics
- Casting in C++
  - Performs fresh inference of types without change of value
  - Performs fresh inference of types with change of value
    - Using implicit computation
    - Using explicit (user-defined) computation
  - Preserves type information in all contexts
  - Provides clear semantics through cast operators:
    - `const_cast`
    - `static_cast`
    - `reinterpret_cast`
    - `dynamic_cast`
  - Cast operators can be `grep`-ed (searched by cast operator name) in source
  - C-Style cast must be avoided in C++

Cast Operators

- A cast operator takes an expression of source type (*implicit* from the expression) and converts it to an expression of target type (*explicit* in the operator) following the semantics of the operator
- Use of cast operators increases robustness by generating errors in static or dynamic time

Cast Operators

- `const_cast` operator: `const_cast<type>(expr)`
  - Explicitly *overrides* `const` *and/or volatile* in a cast
  - Usually *does not perform computation or change value*
- `static_cast` operator: `static_cast<type>(expr)`
  - Performs a *non-polymorphic cast*
  - Usually *performs computation to change value* – implicit or user-defined
- `reinterpret_cast` operator: `reinterpret_cast<type>(expr)`
  - Casts between *unrelated pointer types* or *pointer and integer*
  - *Does not perform computation yet reinterprets value*
- `dynamic_cast` operator: `dynamic_cast<type>(expr)`
  - Performs a *run-time cast* that verifies the validity of the cast
  - *Performs pre-defined computation*, sets null or throws exception

const_cast Operator

- const_cast converts between types with different cv-qualification
- Only const_cast may be used to cast away (remove) const-ness or volatility
- Usually does not perform computation or change value

const_cast Operator


```
#include <iostream>
using namespace std;

class A { int i_;
public: A(int i) : i_(i) { }
    int get() const { return i_; }
    void set(int j) { i_ = j; }
};
void print(char * str) { cout << str; }

int main() {
    const char * c = "sample text";
    // print(c); // error: 'void print(char *)': cannot convert argument 1 from 'const char *' to 'char *'

    print(const_cast<char *>(c)); // Okay

    const A a(1);
    a.get();

    // a.set(5); // error: 'void A::set(int)': cannot convert 'this' pointer from 'const A' to 'A &'

    const_cast<A&>(a).set(5); // Okay

    // const_cast<A>(a).set(5); // error: 'const_cast': cannot convert from 'const A' to 'A'
}
```

const_cast Operator vis-a-vis C-Style Cast


```
#include <iostream>
using namespace std;

class A { int i_;
public: A(int i) : i_(i) { }
    int get() const { return i_; }
    void set(int j) { i_ = j; }
};
void print(char * str) { cout << str; }

int main() {
    const char * c = "sample text";

    // print(const_cast<char *>(c));
    print((char *)(c));          // C-Style Cast

    const A a(1);

    // const_cast<A&>(a).set(5);
    ((A&)a).set(5);              // C-Style Cast

    // const_cast<A>(a).set(5); // error: 'const_cast': cannot convert from 'const A' to 'A'
    ((A)a).set(5);               // C-Style Cast
}
```

const_cast Operator

```
#include <iostream>
struct type { type(): i(3) { }
    void m1(int v) const {
        //this->i = v; // error C3490: 'i' cannot be modified -- accessed through a const object
        const_cast<type*>(this)->i = v; // Okay as long as the type object isn't const
    }
    int i;
};
int main() { int i = 3;                                    // i is not declared const
    const int& cref_i = i; const_cast<int&>(cref_i) = 4; // Okay: modifies i
    std::cout << "i = " << i << '\n';

    type t; // note, if this is const type t;, then t.m1(4); may be undefined behavior
    t.m1(4);
    std::cout << "type::i = " << t.i << '\n';

    const int j = 3;                          // j is declared const
    int* pj = const_cast<int*>(&j); *pj = 4;  // undefined behavior! Value of j and *pj may differ
    std::cout << j << " " << *pj << std::endl;

    void (type::*mfp)(int) const = &type::m1; // pointer to member function
    //const_cast<void(type::*)(int)>(mfp);    // error C2440: 'const_cast': cannot convert from
                                              // 'void (__thiscall type::* )(int) const' to
                                              // 'void (__thiscall type::* )(int)' const_cast does not work
}                                             // on function pointers
```

Output:
```
i = 4
type::i = 4
3 4
```

Module Summary

- Understood casting in C and C++
- Explained cast operators in C++ and discussed the evils of C-style casting
- Studied `const_cast` with examples

Programming in Modern C++

Module M33: Type Casting & Cast Operators: Part 2

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional

Module Recap

- Understood casting in C and C++
- Explained cast operators in C++ and discussed the evils of C-style casting
- Studied `const_cast` with examples

Module Objectives

- Understand casting in C and C++
- Understand `static_cast`, and `reinterpret_cast` operators

Module Outline

Cast Operators

Casting in C and C++: RECAP (Module 32)

- Casting in C
  - Implicit cast
  - Explicit C-Style cast
  - Loses type information in several contexts
  - Lacks clarity of semantics
- Casting in C++
  - Performs fresh inference of types without change of value
  - Performs fresh inference of types with change of value
    - Using implicit computation
    - Using explicit (user-defined) computation
  - Preserves type information in all contexts
  - Provides clear semantics through cast operators:
    - `const_cast`
    - `static_cast`
    - `reinterpret_cast`
    - `dynamic_cast`
  - Cast operators can be grep-ed (searched by cast operator name) in source
  - C-Style cast must be avoided in C++

static_cast Operator

- `static_cast` performs all conversions allowed implicitly (not only those with pointers to classes), and also the opposite of these. It can:
  - Convert from `void*` to any pointer type
  - Convert integers, floating-point values to `enum` types
  - Convert one `enum` type to another `enum` type
- `static_cast` can perform conversions between pointers to related classes:
  - Not only up-casts, but also down-casts
  - No checks are performed during run-time to guarantee that the object being converted is in fact a full object of the destination type
- Additionally, `static_cast` can also perform the following:
  - Explicitly call a single-argument constructor or a conversion operator – The User-Defined Cast
  - Convert to rvalue references
  - Convert `enum` values into integers or floating-point values
  - Convert any type to `void`, evaluating and discarding the value

static_cast Operator: Built-in Types


```
#include <iostream>
using namespace std;
int main() { // Built-in Types
    int i = 2; long j; double d = 3.7; int *pi = &i; double *pd = &d; void *pv = 0;

    i = d;                      // implicit -- warning
    i = static_cast<int>(d);    // static_cast -- okay
    i = (int)d;                 // C-style -- okay

    d = i;                      // implicit -- okay
    d = static_cast<double>(i); // static_cast -- okay
    d = (double)i;              // C-style -- okay

    pv = pi;                    // implicit -- okay
    pi = pv;                    // implicit -- error
    pi = static_cast<int*>(pv); // static_cast -- okay
    pi = (int*)pv;              // C-style -- okay

    j = pd;                     // implicit -- error
    j = static_cast<long>(pd);  // static_cast -- error
    j = (long)pd;               // C-style -- okay: sizeof(long) = 8 = sizeof(double*)
                                // RISKY: Should use reinterpret_cast
    i = (int)pd;                // C-style -- error: sizeof(int) = 4 != 8 = sizeof(double*)
                                // Refer to Module 26 for details
}
```

static_cast Operator: Class Hierarchy


```
#include <iostream>
using namespace std;

// Class Hierarchy
class A { };
class B: public A { };

int main() {
    A a;
    B b;

    // UPCAST
    A *p = 0;
    p = &b;                     // implicit -- okay
    p = static_cast<A*>(&b); // static_cast -- okay
    p = (A*)&b;                 // C-style -- okay

    // DOWNCAST
    B *q = 0;
    q = &a;                     // implicit -- error
    q = static_cast<B*>(&a); // static_cast -- okay: RISKY: Should use dynamic_cast
    q = (B*)&a;                 // C-style -- okay
}
```

static_cast Operator: Pitfall


```
class Window { public:
    virtual void onResize(); ...
}
class SpecialWindow: public Window { // derived class
public:
    virtual void onResize() { // derived onResize impl;
        static_cast<Window>(*this).onResize(); // cast *this to Window, then call its onResize;
            // this doesn't work!

        ... // do SpecialWindow-specific stuff
    }
    ...
};
```

Slices the object, creates a temporary and calls the method!

```
class SpecialWindow: public Window { // derived class
public:
    virtual void onResize() { // derived onResize impl;
        Window::onResize();   // Direct call works

        ... // do SpecialWindow-specific stuff
    }
    ...
};
```

static_cast Operator: Unrelated Classes


```
#include <iostream>
using namespace std;

// Un-related Types
class B;
class A { public:


};
class B { };

int main() {
    A a; B b;
    int i = 5;

    // B ==> A
    a = b;                  // error
    a = static_cast<A>(b); // error
    a = (A)b;               // error

    // int ==> A
    a = i;                  // error
    a = static_cast<A>(i); // error
    a = (A)i;               // error
}
```

```
#include <iostream>
using namespace std;

// Un-related Types
class B;
class A { public:
    A(int i = 0) { cout << "A::A(i)\n"; }
    A(const B&) { cout << "A::A(B&)\n"; }
};
class B { };

int main() {
    A a; B b;
    int i = 5;

    // B ==> A
    a = b;                  // Uses A::A(B&)
    a = static_cast<A>(b); // Uses A::A(B&)
    a = (A)b;               // Uses A::A(B&)

    // int ==> A
    a = i;                  // Uses A::A(int)
    a = static_cast<A>(i); // Uses A::A(int)
    a = (A)i;               // Uses A::A(int)
}
```

static_cast Operator: Unrelated Classes


```
#include <iostream>
using namespace std;

// Un-related Types
class B;
class A { int i_; public:


};
class B { public:

};
int main() { A a; B b; int i = 5;

    // B ==> A
    a = b;                  // error
    a = static_cast<A>(b); // error
    a = (A)b;               // error

    // A ==> int
    i = a;                    // error
    i = static_cast<int>(a); // error
    i = (int)a;               // error
}
```

```
#include <iostream>
using namespace std;

// Un-related Types
class B;
class A { int i_; public:
    A(int i = 0) : i_(i) { cout << "A::A(i)\n"; }
    operator int() { cout << "A::operator int()\n"; return i_; }
};
class B { public:
    operator A() { cout << "B::operator A()\n"; return A(); }
};
int main() { A a; B b; int i = 5;

    // B ==> A
    a = b;                  // B::operator A()
    a = static_cast<A>(b); // B::operator A()
    a = (A)b;               // B::operator A()

    // A ==> int
    i = a;                    // A::operator int()
    i = static_cast<int>(a); // A::operator int()
    i = (int)a;               // A::operator int()
}
```

reinterpret_cast Operator

- `reinterpret_cast` converts *any pointer type* to *any other pointer type*, even of unrelated classes
- The operation result is a simple binary copy of the value from one pointer to the other
- All pointer conversions are allowed: neither the content pointed nor the pointer type itself is checked
- It can also cast pointers to or from integer types
- The format in which this integer value represents a pointer is platform-specific
- The only guarantee is that a pointer cast to an integer type large enough to fully contain it (such as `intptr_t`), is guaranteed to be able to be cast back to a valid pointer (Refer to Module 26)
- The conversions that can be performed by `reinterpret_cast` but not by `static_cast` are low-level operations based on reinterpreting the binary representations of the types, which on most cases results in code which is system-specific, and thus non-portable

reinterpret_cast Operator


```
#include <iostream>
using namespace std;

class A { };
class B { };

int main() {
    long i = 2;
    double d = 3.7;
    double *pd = &d;

    i = pd;                                  // implicit -- error
    i = reinterpret_cast<long>(pd);          // reinterpret_cast -- okay
    i = (long)pd;                            // C-style -- okay
    cout << pd << " " << i << endl;

    A *pA;
    B *pB;

    pA = pB;                                 // implicit -- error
    pA = reinterpret_cast<A*>(pB);           // reinterpret_cast -- okay
    pA = (A*)pB;                             // C-style -- okay
}
```

Module Summary

- Studied `static_cast`, and `reinterpret_cast` with examples

Programming in Modern C++

Module M34: Type Casting & Cast Operators: Part 3

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional

Module Recap

- Studied `static_cast`, and `reinterpret_cast` with examples

Module Objectives

- Understand casting in C and C++
- Understand `dynamic_cast` and `typeid` operators
- Understand RTTI

Module Outline

Cast Operators

Casting in C and C++: RECAP (Module 32)

- Casting in C
  - Implicit cast
  - Explicit C-Style cast
  - Loses type information in several contexts
  - Lacks clarity of semantics
- Casting in C++
  - Performs fresh inference of types without change of value
  - Performs fresh inference of types with change of value
    - Using implicit computation
    - Using explicit (user-defined) computation
  - Preserves type information in all contexts
  - Provides clear semantics through cast operators:
    - `const_cast`
    - `static_cast`
    - `reinterpret_cast`
    - `dynamic_cast`
  - Cast operators can be grep-ed (searched by cast operator name) in source
  - C-Style cast must be avoided in C++

dynamic_cast Operator

- `dynamic_cast` can only be used with *pointers* and *references* to classes (or with `void*`)
- Its purpose is to ensure that the result of the type conversion points to a valid complete object of the destination pointer type
- This naturally includes pointer upcast (converting from pointer-to-derived to pointer-to-base), in the same way as allowed as an implicit conversion
- But `dynamic_cast` can also downcast (convert from pointer-to-base to pointer-to-derived) polymorphic classes (those with virtual members) if-and-only-if the pointed object is a valid complete object of the target type
- If the pointed object is not a valid complete object of the target type, `dynamic_cast` returns a null pointer
- If `dynamic_cast` is used to convert to a reference type and the conversion is not possible, an exception of type `bad_cast` is thrown instead
- `dynamic_cast` can also perform the other implicit casts allowed on pointers: casting null pointers between pointers types (even between unrelated classes), and casting any pointer of any type to a `void*` pointer

dynamic_cast Operator: Pointers


```
#include <iostream>
using namespace std;
class A { public: virtual ~A() { } };
class B: public A { };
class C { public: virtual ~C() { } };
int main() { A a; B b; C c;
    B* pB = &b;  A *pA = dynamic_cast<A*>(pB);
    cout << pB << " casts to " << pA << ": Up-cast: Valid" << endl;

    pA = &b; pB = dynamic_cast<B*>(pA);
    cout << pA << " casts to " << pB << ": Down-cast: Valid" << endl;

    pA = &a; pB = dynamic_cast<B*>(pA);
    cout << pA << " casts to " << pB << ": Down-cast: Invalid" << endl;

    pA = (A*)&c; C *pC = dynamic_cast<C*>(pA);
    cout << pA << " casts to " << pC << ": Unrelated-cast: Invalid" << endl;

    pA = 0; pC = dynamic_cast<C*>(pA);
    cout << pA << " casts to " << pC << ": Unrelated-cast: Valid for null" << endl;

    pA = &a; void *pV = dynamic_cast<void*>(pA);
    cout << pA << " casts to " << pV << ": Cast-to-void: Valid" << endl;

    // pA = dynamic_cast<A*>(pV); // error: 'void *': invalid expression type for dynamic_cast
}
```

```
00EFFCA8 casts to 00EFFCA8: Up-cast: Valid
00EFFCA8 casts to 00EFFCA8: Down-cast: Valid
00EFFCB4 casts to 00000000: Down-cast: Invalid
00EFFC9C casts to 00000000: Unrelated-cast: Invalid
00000000 casts to 00000000: Unrelated: Valid for null
00EFFCB4 casts to 00EFFCB4: Cast-to-void: Valid
```

dynamic_cast Operator: References

```
#include <iostream>
#include <typeinfo>
using namespace std;
class A { public: virtual ~A() { } };
class B: public A { };
class C { public: virtual ~C() { } };

int main() { A a; B b; C c;
    try { B &rB1 = b;
        A &rA2 = dynamic_cast<A&>(rB1);
        cout << "Up-cast: Valid" << endl;

        A &rA3 = b;
        B &rB4 = dynamic_cast<B&>(rA3);
        cout << "Down-cast: Valid" << endl;

        try { A &rA5 = a;
            B &rB6 = dynamic_cast<B&>(rA5);
        } catch (bad_cast e) { cout << "Down-cast: Invalid: " << e.what() << endl; }

        try { A &rA7 = (A&)c;
            C &rC8 = dynamic_cast<C&>(rA7);
        } catch (bad_cast e) { cout << "Unrelated-cast: Invalid: " << e.what() << endl; }
    } catch (bad_cast e) { cout << "Bad-cast: " << e.what() << endl; }
}
```

MSVC++

```
Up-cast: Valid
Down-cast: Valid
Down-cast: Invalid: Bad dynamic_cast!
Unrelated-cast: Invalid: Bad dynamic_cast!
```

Onlinegdb

```
Up-cast: Valid
Down-cast: Valid
Down-cast: Invalid: std::bad_cast
Unrelated-cast: Invalid: std::bad_cast
```

typeid Operator

typeid Operator

- `typeid` operator is used where the dynamic type of a polymorphic object must be known and for static type identification
- `typeid` operator can be applied on a type or an expression
- `typeid` operator returns `const std::type_info`. The major members are:
  - `operator==, operator!=`: checks whether the objects refer to the same type
  - `name`: implementation-defined name of the type
- `typeid` operator works for polymorphic type only (as it uses RTTI – virtual function table)
- If the polymorphic object is bad, the `typeid` throws `bad_typeid` exception

Using typeid Operator: Polymorphic Hierarchy

```
#include <iostream>
#include <typeinfo>
using namespace std;

// Polymorphic Hierarchy
class A { public: virtual ~A() { } };
class B : public A { };

int main() {
    A a;
    cout << typeid(a).name() << ": " << typeid(&a).name() << endl; // Static
    A *p = &a;
    cout << typeid(p).name() << ": " << typeid(*p).name() << endl; // Dynamic

    B b;
    cout << typeid(b).name() << ": " << typeid(&b).name() << endl; // Static
    p = &b;
    cout << typeid(p).name() << ": " << typeid(*p).name() << endl; // Dynamic

    A &r1 = a;
    A &r2 = b;
    cout << typeid(r1).name() << ": " << typeid(r2).name() << endl; // Dynamic
}
```

MSVC++	Onlinegdb
class A: class A *	1A: P1A
class A *: class A	P1A: 1A
class B: class B *	1B: P1B
class A *: class B	P1A: 1B
class A: class B	1A: 1B

Using typeid Operator: Polymorphic Hierarchy: Staff Salary Application

```
#include <iostream>                  | MSVC++                                | Onlinegdb
#include <string>                    |                                       |
#include <typeinfo>                  | class Engineer *: class Engineer      | P8Engineer: 8Engineer
using namespace std;                 | class Engineer *: class Manager       | P8Engineer: 7Manager
                                     | class Engineer *: class Director      | P8Engineer: 8Director
class Engineer { protected: string name_;
public: Engineer(const string& name) : name_(name) { }
    virtual void ProcessSalary() { cout << name_ << ": Process Salary for Engineer" << endl; }
};
class Manager : public Engineer { Engineer *reports_[10];
public: Manager(const string& name) : Engineer(name) { }
    void ProcessSalary() { cout << name_ << ": Process Salary for Manager" << endl; }
};
class Director : public Manager { Manager *reports_[10];
public: Director(const string& name) : Manager(name) { }
    void ProcessSalary() { cout << name_ << ": Process Salary for Director" << endl; }
};
int main() {
    Engineer e("Rohit"); Manager m("Kamala"); Director d("Ranjana");
    Engineer *staff[] = { &e, &m, &d };
    for (int i = 0; i < sizeof(staff) / sizeof(Engineer*); ++i) {
        cout << typeid(staff[i]).name() << ": " << typeid(*staff[i]).name() << endl;
    }
}
```

Using typeid Operator: Non-Polymorphic Hierarchy

```
#include <iostream>                  | MSVC++                  | Onlinegdb
#include <typeinfo>                  |                         |
using namespace std;                 | class X: class X *      | 1X: P1X
                                     | class X *: class X      | P1X: 1X
// Non-Polymorphic Hierarchy         | class Y: class Y *      | 1Y: P1Y
class X { };                         | class X *: class X      | P1X: 1X
class Y : public X { };              | class X: class X        | 1X: 1X

int main() {
    X x;
    cout << typeid(x).name() << ": " << typeid(&x).name() << endl; // Static
    X *q = &x;
    cout << typeid(q).name() << ": " << typeid(*q).name() << endl; // Dynamic

    Y y;
    cout << typeid(y).name() << ": " << typeid(&y).name() << endl; // Static
    q = &y;
    cout << typeid(q).name() << ": " << typeid(*q).name() << endl; // Dynamic -- FAILS

    X &r1 = x; X &r2 = y;
    cout << typeid(r1).name() << ": " << typeid(r2).name() << endl; // Dynamic
}
```

Using typeid Operator: bad_typeid Exception


```
#include <iostream>
#include <typeinfo>
using namespace std;

class A { public: virtual ~A() { } };
class B : public A { };

int main() { A *pA = new A;
    try {
        cout << typeid(pA).name() << endl;
        cout << typeid(*pA).name() << endl;
    } catch (const bad_typeid& e)
        { cout << "caught " << e.what() << endl; }
    delete pA;
    try {
        cout << typeid(pA).name() << endl;
        cout << typeid(*pA).name() << endl;
    } catch (const bad_typeid& e) { cout << "caught " << e.what() << endl; }
    pA = 0;
    try {
        cout << typeid(pA).name() << endl;
        cout << typeid(*pA).name() << endl;
    }
    catch (const bad_typeid& e) { cout << "caught " << e.what() << endl; }
}
```

MSVC++

```
class A *
class A
class A *
caught Access violation - no RTTI data!
class A *
caught Attempted a typeid of NULL pointer!
```

Onlinegdb

```
P1A
1A
P1A
```

Run-Time Type Information (RTTI)

Run-Time Type Information (RTTI)

- *Run-Time Type Information* or *Run-Time Type Identification* (RTTI) exposes information about an object's data type at runtime
- RTTI is a specialization of a more general concept called *Type Introspection*
  - *Type Introspection* helps to examine the type or properties of an object at runtime
  - Introspection should not be confused with *reflection*, which is the ability for a program to manipulate the values, metadata, properties, and functions of an object at runtime
- RTTI can be used to do safe typecasts, using the `dynamic_cast<>` operator, and to manipulate type information at runtime, using the `typeid` operator and `std::type_info` class
- RTTI is available only *polymorphic* classes, with at least one virtual method (destructor)
- Some compilers have *flags to disable RTTI* to reduce the size of the application
- `typeid` keyword is used to determine the class of an object at run time. It returns a reference to `std::type_info` object, which exists until the end of the program
- The use of `typeid`, in a non-polymorphic context, is often preferred over `dynamic_cast<class_type>` for efficiency
- Objects of class `std::bad_typeid` are thrown when the expression for `typeid` is the result of applying the unary * operator on a null pointer

Module Summary

- Understood casting at run-time
- Studied `dynamic_cast` with examples
- Understood RTTI and `typeid` operator

Programming in Modern C++

Module M35: Multiple Inheritance

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional

Module Recap

- Understood casting at run-time
- Studied `dynamic_cast` with examples
- Understood RTTI and `typeid` operator

Module Objectives

- Understand Multiple Inheritance in C++

Module Outline

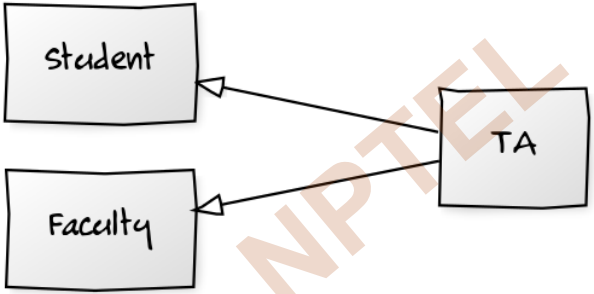
Multiple Inheritance in C++

Source:

- Is inheritance bad practice in OOP?, quora, 2019

Multiple Inheritance in C++: Hierarchy

- TA **ISA** Student; TA **ISA** Faculty

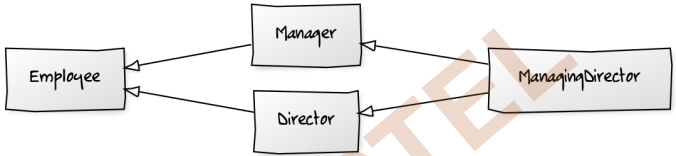



```
class Student;                              // Base Class = Student
class Faculty;                              // Base Class = Faculty
class TA: public Student, public Faculty; // Derived Class = TA
```

- TA inherits properties and operations of both Student as well as Faculty

Multiple Inheritance in C++: Hierarchy

- Manager **ISA** Employee, Director **ISA** Employee, ManagingDirector **ISA** Manager, ManagingDirector **ISA** Director

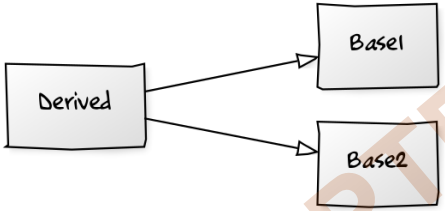



```
class Employee;                                          // Base Class = Employee -- Root
class Manager: public Employee;                          // Derived Class = Manager
class Director: public Employee;                         // Derived Class = Director
class ManagingDirector: public Manager, public Director; // Derived Class = ManagingDirector
```

- Manager inherits properties and operations of Employee
- Director inherits properties and operations of Employee
- ManagingDirector inherits properties and operations of both Manager as well as Director
- ManagingDirector, by transitivity, inherits properties and operations of Employee
- **Multiple inheritance hierarchy usually has a common base class**
- This is known as the **Diamond Hierarchy**

Multiple Inheritance in C++: Semantics

- Derived **ISA** Base1, Derived **ISA** Base2




```
class Base1;                                  // Base Class = Base1
class Base2;                                  // Base Class = Base2
class Derived: public Base1, public Base2;    // Derived Class = Derived
```

- Use keyword `public` after class name to denote inheritance
- Name of the Base class follow the keyword
- There may be more than two base classes
- `public` and `private` inheritance may be mixed

Multiple Inheritance in C++: Semantics

- Data Members
  - Derived class *inherits all* data members of *all* Base classes
  - Derived class may *add* data members of its own
- Member Functions
  - Derived class *inherits all* member functions of *all* Base classes
  - Derived class may *override* a member function of *any* Base class by *redefining* it with the same signature
  - Derived class may *overload* a member function of *any* Base class by *redefining* it with the *same name*; but *different signature*
- Access Specification
  - Derived class *cannot access* private members of *any* Base class
  - Derived class *can access* protected members of *any* Base class
- Construction-Destruction
  - A *constructor* of the Derived class *must first* call *all constructor*s of the Base classes to construct the Base class instances of the Derived class – Base class *constructor*s are called in *listing order*
  - The *destructor* of the Derived class *must* call the *destructor*s of the Base classes to destruct the Base class instances of the Derived class

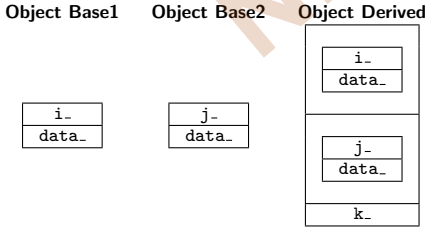
Multiple Inheritance in C++: Data Members and Object Layout

- Data Members
  - Derived class *inherits all* data members of *all* Base classes
  - Derived class may *add* data members of its own
- Object Layout
  - Derived class *layout* contains instances of *each* Base class
  - Further, Derived class layout will have data members of its own
  - C++ does not guarantee the relative position of the Base class instances and Derived class members

Multiple Inheritance in C++: Data Members and Object Layout

```
class Base1 { protected:
    int i_, data_;
public: // ...
};
class Base2 { protected:
    int j_, data_;
public: // ...
};
class Derived: public Base1, public Base2 { // Multiple inheritance
    int k_;
public: // ...
};
```

Object Layout



- `Object Derived` has two `data_` members!
- Ambiguity to be resolved with base class name: `Base1::data_` & `Base2::data_`

Multiple Inheritance in C++: Member Functions – Overrides and Overloads

- Derived **ISA** Base1, Base2
- Member Functions
  - Derived class *inherits all* member functions of *all* Base classes
  - Derived class may *override* a member function of *any* Base class by *redefining* it with the same signature
  - Derived class may *overload* a member function of *any* Base class by *redefining* it with the *same name*; but *different signature*
- Static Member Functions
  - Derived class *does not inherit* the static member functions of *any* Base class
- Friend Functions
  - Derived class *does not inherit* the friend functions of *any* Base class

Multiple Inheritance in C++: Member Functions – Overrides and Overloads


```
class Base1 { protected: int i_, data_;
public: Base1(int a, int b): i_(a), data_(b) { }
    void f(int) { cout << "Base1::f(int) \n"; }
    void g() { cout << "Base1::g() \n"; }
};
class Base2 { protected: int j_, data_;
public: Base2(int a, int b): j_(a), data_(b) { }
    void h(int) { cout << "Base2::h(int) \n"; }
};
class Derived: public Base1, public Base2 { int k_;
public: Derived(int x, int y, int u, int v, int z): Base1(x, y), Base2(u, v), k_(z) { }
    void f(int) { cout << "Derived::f(int) \n"; }       // -- Overridden Base1::f(int)
    // -- Inherited Base1::g()
    void h(string) { cout << "Derived::h(string) \n"; } // -- Overloaded Base2:: h(int)
    void e(char) { cout << "Derived::e(char) \n"; }     // -- Added Derived::e(char)
};

Derived c(1, 2, 3, 4, 5);

c.f(5);      // Derived::f(int)   -- Overridden Base1::f(int)
c.g();       // Base1::g()        -- Inherited Base1::g()
c.h("ppd"); // Derived::h(string) -- Overloaded Base2:: h(int)
c.e('a');    // Derived::e(char)  -- Added Derived::e(char)
```

Inheritance in C++: Member Functions – using for Name Resolution

Ambiguous Calls

```
class Base1 { public:
    Base1(int a, int b);
    void f(int) { cout << "Base1::f(int) "; }
    void g() { cout << "Base1::g() "; }
};
class Base2 { public:
    Base2(int a, int b);
    void f(int) { cout << "Base2::f(int) "; }
    void g(int) { cout << "Base2::g(int) "; }
};
class Derived: public Base1, public Base2 {
public: Derived(int x, int y, int u, int v, int z);


};
Derived c(1, 2, 3, 4, 5);

c.f(5); // Base1::f(int) or Base2::f(int)?
c.g(5); // Base1::g() or Base2::g(int)?
c.f(3); // Base1::f(int) or Base2::f(int)?
c.g();  // Base1::g() or Base2::g(int)?
```

Unambiguous Calls

```
class Base1 { public:
    Base1(int a, int b);
    void f(int) { cout << "Base1::f(int) "; }
    void g() { cout << "Base1::g() "; }
};
class Base2 { public:
    Base2(int a, int b);
    void f(int) { cout << "Base2::f(int) "; }
    void g(int) { cout << "Base2::g(int) "; }
};
class Derived: public Base1, public Base2 {
public: Derived(int x, int y, int u, int v, int z);
    using Base1::f; // Hides Base2::f
    using Base2::g; // Hides Base1::g
};
Derived c(1, 2, 3, 4, 5);

c.f(5);         // Base1::f(int)
c.g(5);         // Base2::g(int)
c.Base2::f(3); // Base2::f(int)
c.Base1::g();  // Base1::g()
```

- Overload resolution does not work between Base1::g() and Base2::g(int)
- using hides other candidates; Explicit use of base class name can resolve (weak solution)

Multiple Inheritance in C++: Access Members of Base: `protected` Access

- Access Specification
  - Derived class *cannot access* private members of *any* Base class
  - Derived class *can access* protected members of *any* Base class

Multiple Inheritance in C++: Constructor & Destructor

- Constructor-Destructor
  - Derived class *inherits all* Constructors and Destructor of Base classes (but in a different semantics)
  - Derived class *cannot overload* a Constructor or *cannot override* the Destructor of *any* Base class
- Construction-Destruction
  - A *constructor* of the Derived class *must first* call *all constructor*s of the Base classes to construct the Base class instances of the Derived class
  - Base class *constructor*s are called in *listing order*
  - The *destructor* of the Derived class *must* call the *destructor*s of the Base classes to destruct the Base class instances of the Derived class

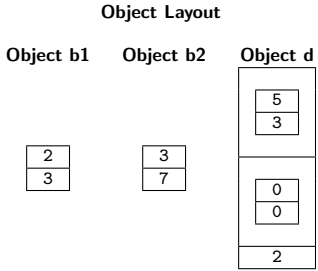
Multiple Inheritance in C++: Constructor & Destructor

```
class Base1 { protected: int i_; int data_;
public: Base1(int a, int b): i_(a), data_(b) { cout << "Base1::Base1() "; }
    ~Base1() { cout << "Base1::~Base1() "; }
};

class Base2 { protected: int j_; int data_;
public: Base2(int a = 0, int b = 0): j_(a), data_(b) { cout << "Base2::Base2() "; }
    ~Base2() { cout << "Base2::~Base2() "; }
};

class Derived: public Base1, public Base2 { int k_;
public: Derived(int x, int y, int z):
            Base1(x, y), k_(z) { cout << "Derived::Derived() "; }
            // Base1::Base1 explicit, Base2::Base2 default
    ~Derived() { cout << "Derived::~Derived() "; }
};

Base1 b1(2, 3);
Base2 b2(3, 7);
Derived d(5, 3, 2);
```

Multiple Inheritance in C++: Object Lifetime

```
class Base1 { protected: int i_; int data_;
public: Base1(int a, int b): i_(a), data_(b)
        { cout << "Base1::Base1() " << i_ << ' ' << data_ << endl; }
    ~Base1() { cout << "Base1::~Base1() " << i_ << ' ' << data_ << endl; }
};
class Base2 { protected: int j_; int data_;
public: Base2(int a = 0, int b = 0): j_(a), data_(b)
        { cout << "Base2::Base2() " << j_ << ' ' << data_ << endl; }
    ~Base2() { cout << "Base2::~Base2() " << j_ << ' ' << data_ << endl; }
};
class Derived: public Base1, public Base2 { int k_; public:
    Derived(int x, int y, int z): Base1(x, y), k_(z)
        { cout << "Derived::Derived() " << k_ << endl; }
        // Base1::Base1 explicit, Base2::Base2 default
    ~Derived() { cout << "Derived::~Derived() " << k_ << endl; }
};

Derived d(5, 3, 2);
```

Construction O/P

```
Base1::Base1(): 5, 3  // Obj. d.Base1
Base2::Base2(): 0, 0  // Obj. d.Base2
Derived::Derived(): 2 // Obj. d
```

Destruction O/P

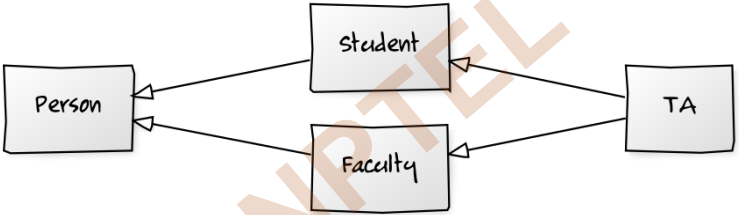
```
Derived::~Derived(): 2 // Obj. d
Base2::~Base2(): 0, 0  // Obj. d.Base2
Base1::~Base1(): 5, 3  // Obj. d.Base1
```

- First construct base class objects, then derived class object
- First destruct derived class object, then base class objects

Diamond Problem

Multiple Inheritance in C++: Diamond Problem

- Student **ISA** Person
- Faculty **ISA** Person
- TA **ISA** Student; TA **ISA** Faculty




```
class Person;                                // Base Class = Person -- Root
class Student: public Person;                // Base / Derived Class = Student
class Faculty: public Person;                // Base / Derived Class = Faculty
class TA: public Student, public Faculty; // Derived Class = TA
```

- Student inherits properties and operations of Person
- Faculty inherits properties and operations of Person
- TA inherits properties and operations of both Student as well as Faculty
- TA, by transitivity, inherits properties and operations of Person

Multiple Inheritance in C++: Diamond Problem


```
#include<iostream>
using namespace std;

class Person { // data members of person
    public: Person(int x)  { cout << "Person::Person(int)" << endl; }
};
class Faculty: public Person { // data members of Faculty
    public: Faculty(int x): Person(x) { cout << "Faculty::Faculty(int)" << endl; }
};
class Student: public Person { // data members of Student
    public: Student(int x): Person(x) { cout << "Student::Student(int)" << endl; }
};
class TA: public Faculty, public Student {
    public: TA(int x): Student(x), Faculty(x) { cout << "TA::TA(int)" << endl; }
};
int main() { TA ta(30);
}
```

```
Person::Person(int)
Faculty::Faculty(int)
Person::Person(int)
Student::Student(int)
TA::TA(int)
```

- **Two instances of base class object (`Person`) in a `TA` object!**

Multiple Inheritance in C++: virtual Inheritance – virtual Base Class


```
#include<iostream>
using namespace std;
class Person { // data members of person
    public: Person(int x) { cout << "Person::Person(int)" << endl; }
    Person() { cout << "Person::Person()" << endl; } // Default ctor for virtual inheritance
};
class Faculty: virtual public Person { // data members of Faculty
    public: Faculty(int x): Person(x) { cout << "Faculty::Faculty(int)" << endl; }
};
class Student: virtual public Person { // data members of Student
    public: Student(int x): Person(x) { cout << "Student::Student(int)" << endl; }
};
class TA: public Faculty, public Student {
    public: TA(int x): Student(x), Faculty(x) { cout << "TA::TA(int)" << endl; }
};
int main()  { TA ta(30); }
```

```
Person::Person()
Faculty::Faculty(int)
Student::Student(int)
TA::TA(int)
```

- Introduce a default constructor for root base class `Person`
- Prefix every inheritance of `Person` with `virtual`
- **Only one instance of base class object (`Person`) in a `TA` object!**

Multiple Inheritance in C++: `virtual` Inheritance with Parameterized Ctor


```
#include<iostream>
using namespace std;

class Person {
    public: Person(int x) { cout << "Person::Person(int)" << endl; }
    Person() { cout << "Person::Person()" << endl; }
};
class Faculty: virtual public Person {
    public: Faculty(int x): Person(x) { cout << "Faculty::Faculty(int)" << endl; }
};
class Student: virtual public Person {
    public: Student(int x): Person(x) { cout << "Student::Student(int)" << endl; }
};
class TA: public Faculty, public Student {
    public: TA(int x): Student(x), Faculty(x), Person(x) { cout << "TA::TA(int)" << endl; }
};
int main() { TA ta(30); }
```

```
Person::Person(int)
Faculty::Faculty(int)
Student::Student(int)
TA::TA(int )
```

- Call parameterized constructor of root base class `Person` from constructor of `TA` class

Multiple Inheritance in C++: Ambiguity


```
#include<iostream>
using namespace std;

class Person {
    public: Person(int x) { cout << "Person::Person(int)" << endl; }
    Person() { cout << "Person::Person()" << endl; }
    virtual ~Person();
    virtual void teach() = 0;
};
class Faculty: virtual public Person {
    public: Faculty(int x): Person(x) { cout << "Faculty::Faculty(int)" << endl; }
    virtual void teach();
};
class Student: virtual public Person {
    public: Student(int x): Person(x) { cout << "Student::Student(int)" << endl; }
    virtual void teach();
};
class TA: public Faculty, public Student {
    public: TA(int x):Student(x), Faculty(x) { cout << "TA::TA(int)" << endl; }
    virtual void teach();
};
```

- **In the absence of `TA::teach()`, which of `Student::teach()` or `Faculty::teach()` should be inherited?**

Multiple Inheritance in C++: Exercise

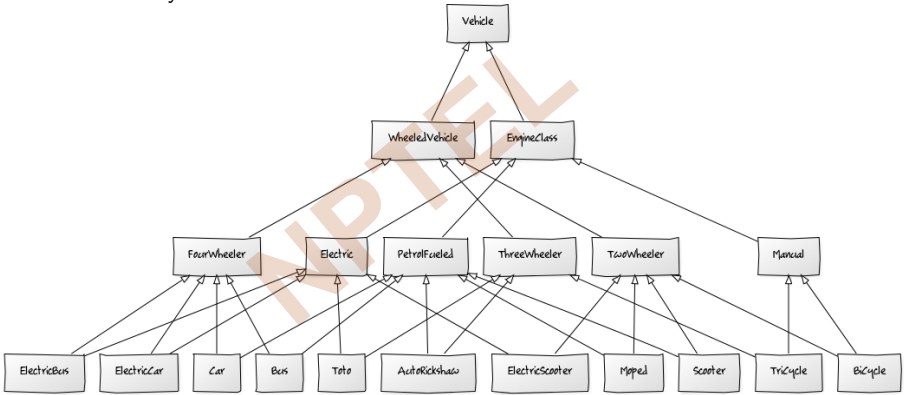

```
class A {
public:
    virtual ~A() { cout << "A::~A()" << endl; }
    virtual void foo() { cout << "A::foo()" << endl; }
};
class B: public virtual A {
public:
    virtual ~B() { cout << "B::~B()" << endl; }
    virtual void foo() { cout << "B::foo()" << endl; }
};
class C: public virtual A {
public:
    virtual ~C() { cout << "C::~C()" << endl; }
    virtual void foobar() { cout << "C::foobar()" << endl; }
};
class D: public B, public C {
public:
    virtual ~D() { cout << "D::~D()" << endl; }
    virtual void foo() { cout << "D::foo()" << endl; }
    virtual void foobar() { cout << "D::foobar()" << endl; }
};
```


- Consider the effect of calling `foo` and `foobar` for various objects and various pointers

Design Choice

Design Choice: Inheritance or Composition

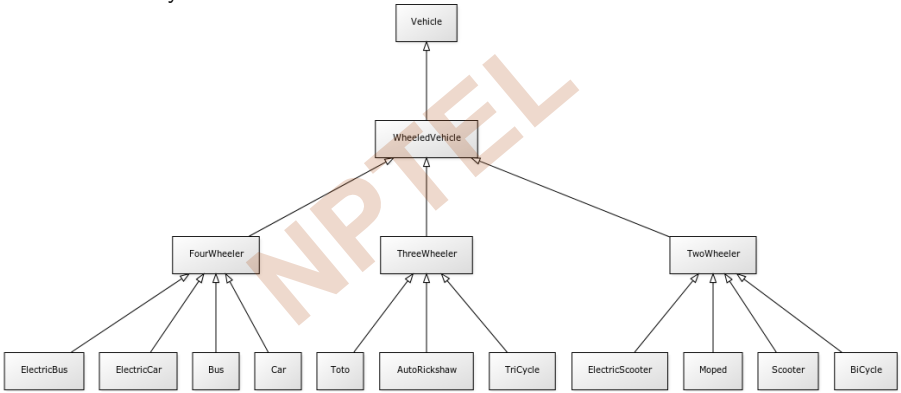
- Vehicle Hierarchy



- Wheeled Hierarchy and Engine Hierarchy interact
- Large number of cross links!
- Multiplicative options make modeling difficult

Design Choice: Inheritance or Composition

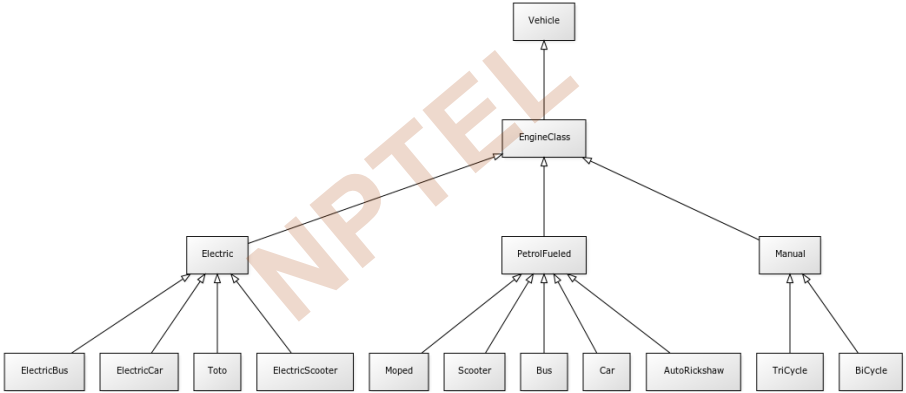
- Vehicle Hierarchy



- Wheeled Hierarchy use Engine as Component
- Linear options to simplify models
- Is this dominant?

Design Choice: Inheritance or Composition

- Vehicle Hierarchy



- Engine Hierarchy use Wheeled as Component
- Linear options to simplify models
- Is this dominant?

Module Summary

- Introduced the Semantics of Multiple Inheritance in C++
- Discussed the Diamond Problem and solution approaches
- Illustrated the design choice between inheritance and composition

Programming in Modern C++

Tutorial T07: How to design a UDT like built-in types?: Part 1: Fraction UDT

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional

Tutorial Objectives

- Understand Building a data type: Fraction type

Tutorial Outline

Data types

Notion of Data types

- Data types in C++ are used to specify the type of the data we use in our programs.
- They are classified under three categories:
  - Built-In or Primitive Data types
  - Derived Data types
  - User-Defined Data type
- **Built-in data types:**
  - Built-in data types are the most basic data types in C++
  - They are predefined and can be used directly in a program
  - **Examples:** `char`, `int`, `float` and `double`
  - Apart from these, we also have `void` and `bool` data types
- **Derived Data types:**
  - Data types that are derived from the built-in types
  - **Examples:** arrays, functions, references and pointers
- **User Defined Type (UDT):**
  - Those are declared & defined by the user using basic data types before using it
  - **Examples:** structures, unions, enumerations and classes

User Defined Types

- Operator overloading helps us *build complete algebra* for UDT's much in the same line as is available for built-in types, called as, *Building data type*
  - **Complex type**: Add (+), Subtract (-), Multiply (*), Divide (/), Conjugate (!), Compare (==, !=, ...), etc.
  - **Fraction type**: Add (+), Subtract (-), Multiply (*), Divide (/), Reduce (unary *), Compare (==, !=, ...), etc.
  - **Matrix type**: Add (+), Subtract (-), Multiply (*), Divide (/), Invert (!), Compare (==, !=, ...), etc.
  - **Set type**: Union (+), Difference (-), Intersection (*), Subset (<, <=), Superset (>, >=), Compare (==, !=), etc.
  - **Direct IO**: read (>>) and write (<<) for all types

Fraction UDT

Design of Fraction UDT

- We intend to design a UDT `Fraction` which can behave like the build-in types like `int`
- The broad tasks involved include:
  - Make a clear statement of the concept of `Fraction`
  - Identify a representation for a `Fraction` object
  - Identify the properties and assertions applicable to all objects
  - Identify the operations for `Fraction` objects
    - Choose appropriate operators to overload for the operations
    - For example `operator+` to add two `Fraction` objects, or `operator<<` to stream a `Fraction` to `cout`
    - *Do not break the natural semantics for the operators*
- While it is possible to design and implement the UDT in one go (once you have acquired some expertise); it is better to go with iterative refinement. That is:
  - Make a design
  - Implement and Test
  - Refine and repeat

Notion of Fraction

- Intuitively fraction is a notation for numbers of the form $\frac{p}{q}$ where p and q are integers, like $\frac{2}{3}$, $\frac{4}{6}$, $\frac{3}{2}$ etc.
  - Fraction representation is *non-unique*: $\frac{2}{3} = \frac{4}{6} = \frac{8}{12} = \frac{-2}{-3}; \cdots, -\frac{2}{3} = \frac{-2}{3} = \frac{2}{-3}$
- For our UDT design, we need *uniqueness of representation*. So let us restrict with the following rules for a fraction $\frac{p}{q}$:
  - q must be *positive*: $q > 0$
  - p and q must be *mutually prime*: $gcd(p,\ q) = 1$

  Such fractions are known as *rational numbers* in mathematics
- Further a fraction $\frac{p}{q}$ is called *proper* if $|\frac{p}{q}| < 1$. It is *improper*, otherwise
  - An *improper fraction* can be written in *mixed fraction format* (assume $p > 0$) where we specify the maximum whole number in the fraction and the remaining proper fraction part:

$$\frac{p}{q} = (p \div q)\frac{p\ \%\ q}{q}$$

  For example, $\frac{17}{3} = 5\frac{2}{3}$

Definition of Fraction

Definition

$\frac{p}{q}$ is a fraction where p and q are integers, $q > 0$, and p and q are mutually prime, that is, $gcd(p, q) = 1$

That is, $p \in \mathcal{Z}, q \in \mathcal{N}$, $gcd(p, q) = 1$, where $\mathcal{Z}$ is the set of integers and $\mathcal{N}$ is the set of natural numbers

p is called the numerator and q is called the denominator

Definition

Any fraction $\frac{p}{q}$ where $gcd(p, q) > 1$, is irreduced and can be reduced to

$$\frac{p}{q} = \frac{p \div gcd(p, q)}{q \div gcd(p, q)}$$

Operations of Fraction

Definition

Reduction:

$$\begin{aligned}\frac{p}{q} &= \frac{p/gcd(p,q)}{q/gcd(p,q)}, \text{ if } gcd(p,q) \neq 1 \\ &= \frac{-p}{-q}, \text{ if } q < 0 \\ &= \frac{0}{1}, \text{ if } p = 0 \\ &= undefined, \text{ if } q = 0\end{aligned}$$

Addition: $\left(\frac{p}{q}\right) + \left(\frac{r}{s}\right) = \frac{p*(lcm(q,s)/q)+r*(lcm(q,s)/s)}{lcm(q,s)}$. Example 1: $\frac{5}{12} + \frac{7}{18} = \frac{5*3+7*2}{36} = \frac{29}{36}$

Subtraction: $\left(\frac{p}{q}\right) - \left(\frac{r}{s}\right) = \left(\frac{p}{q}\right) + \left(\frac{-r}{s}\right)$. Example 2: $\frac{5}{12} - \frac{7}{18} = \frac{5*3+(-7)*2}{36} = \frac{1}{36}$

Multiplication: $\left(\frac{p}{q}\right) * \left(\frac{r}{s}\right) = \frac{p*r}{q*s}$. Example 3: $\frac{5}{12} * \frac{7}{18} = \frac{5*7}{12*18} = \frac{35}{216}$

Division: $\left(\frac{p}{q}\right) / \left(\frac{r}{s}\right) = \frac{p*s}{q*r}$. Example 4: $\frac{5}{12} / \frac{7}{18} = \frac{5*18}{7*12} = \frac{15}{14}$

Modulus: $\left(\frac{p}{q}\right) \% \left(\frac{r}{s}\right) = \frac{p}{q} - \lfloor \left(\frac{p}{q}\right) / \left(\frac{r}{s}\right) \rfloor * \frac{r}{s}$. Example 5: $\frac{5}{12} \% \frac{7}{18} = \frac{5}{12} - \lfloor \frac{15}{14} \rfloor * \frac{7}{18} = \frac{1}{36}$

where $lcm(q,s) = (q*s)/gcd(q,s)$

Rules of Fraction

Fractions obey fives rules of algebra as follows. For two fractions $\frac{p}{q}$ and $\frac{r}{s}$,

Definition

Rule of Invertendo: $\frac{p}{q} = \frac{r}{s} \Rightarrow \frac{q}{p} = \frac{s}{r}$. Use $!\frac{p}{q} = \frac{q}{p}$

Rule of Alternendo: $\frac{p}{q} = \frac{r}{s} \Rightarrow \frac{p}{r} = \frac{q}{s}$

Rule of Componendo: $\frac{p}{q} :: \frac{r}{s} \Rightarrow \frac{p+q}{q} :: \frac{r+s}{s}$. Use $++\frac{p}{q} = \frac{p+q}{q} = \frac{p}{q} + 1$

Rule of Dividendo: $\frac{p}{q} :: \frac{r}{s} \Rightarrow \frac{p-q}{q} :: \frac{r-s}{s}$. Use $--\frac{p}{q} = \frac{p-q}{q} = \frac{p}{q} - 1$

Rule of Componendo & Dividendo: $\frac{p}{q} :: \frac{r}{s} \Rightarrow \frac{p+q}{p-q} :: \frac{r+s}{r-s}$

We define three operations on fractions: Invertendo (`operator!`), Componendo (`operator++`), and Dividendo (`operator--`) to facilitate fraction algebra expressions

Design of Fraction Class

- From the definition, the representation of a `Fraction` can simply be:

```
class Fraction { // Implicit assertion for proper fraction: gcd(|n_|, d_) = 1
    int n_;          // numerator. n_ belongs to Z
    unsigned int d_; // denominator. d_ belongs to N
}
```

- `Fraction` should support the following operation like `int`:
  - Construction, Destruction and Copy Operations
  - Unary Arithmetic Operations: Preserve (Sign), Negate, Componendo, and Dividendo
  - Binary Arithmetic Operations: Add, Subtract, Multiply, Divide, and Mod
  - Binary Relational Operations: Less, LessEq, More, MoreEq, Eq, NotEq
  - IO Operations: Read and Write
- `Fraction` should also support the following extended operation:
  - Invert
  - Convert to `double`
- `Fraction` also need to support the following utilities for convenience:
  - GCD and LCM
  - Reduction (of irreduced fraction to reduced fraction)

Design of Fraction: Interface: Version 1

- Construction, Destruction, and Copy Operations

```
explicit Fraction(int = 1, int = 1);  // Three overloads including a default constructor
~Fraction();                          // No virtual destructor needed
Fraction(const Fraction&);            // Copy constructor
Fraction& operator=(const Fraction&); // Copy assignment operator
```

- IO Operations: Read and Write

```
static void Write(const Fraction&); // Outstreams a fraction to cout in n/d form
static void Read(Fraction&);        // Instreams n & d from cin to construct a fraction
```

- Unary Arithmetic Operations: Negate, Preserve (Sign), Componendo, and Dividendo

```
Fraction  Negate() const;  // Negate. p/q <-- -p/q
Fraction  Preserve() const; // Preserve. p/q <-- p/q
Fraction& Componendo();    // Componendo. p/q <-- p/q + 1
Fraction& Dividendo();     // Dividendo. p/q <-- p/q - 1
```

- Binary Arithmetic Operations: Add, Subtract, Multiply, Divide, and Modulus

```
Fraction Add(const Fraction&) const;      // Generates a result fraction,
Fraction Subtract(const Fraction&) const; // Does not change the current object
Fraction Multiply(const Fraction&) const;
Fraction Divide(const Fraction&) const;
Fraction Modulus(const Fraction&) const;
```

Design of Fraction: Interface: Version 1

- Binary Relational Operations: Less, LessEq, More, MoreEq, Eq, NotEq

```
bool Eq(const Fraction&) const;    // Generates a comparison result
bool NotEq(const Fraction&) const; // Does not change the current object
bool Less(const Fraction&) const;
bool LessEq(const Fraction&) const;
bool More(const Fraction&) const;
bool MoreEq(const Fraction&) const;
```

- Extended Operations: Invert and Convert to `double`

```
Fraction Invert() const; // Inverts a fraction. !(p/q) = q/p
double Double();         // Converts a fraction to a double
```

- Static constant fractions

```
static const Fraction UNITY; // Defines 1/1
static const Fraction ZERO;  // Defines 0/1
```

- Support Functions: gcd, lcm and reduce: Should be `private` - not part of interface

```
static int gcd(int, int); // Finds the gcd for two +ve integers
static int lcm(int, int); // Finds the lcm for two +ve integers
Fraction& Reduce();       // Reduces a fraction
```

Implementation of Fraction: Version 1

- Construction, Destruction, and Copy Operations

```
explicit Fraction(int n = 1, int d = 1): // Three overloads
    n_(d < 0 ? -n : n), d_(d < 0 ? -d : d) // d_ is unsigned int. So no -ve value
{ Reduce(); } // Reduces the fraction
Fraction(const Fraction& f) : n_(f.n_), d_(f.d_) { } // Copy Constructor
~Fraction() { } // No virtual destructor needed
Fraction& operator=(const Fraction& f) { n_ = f.n_; d_ = f.d_; return *this; }
```

- IO Operations: Read and Write

```
static void Write(const Fraction& f) { cout << f.n_;
    if ((f.n_ != 0) && (f.d_ != 1)) cout << "/" << f.d_; // Suppress denominator
                                                        // if n_ == 0 or d_ == 1
}
static void Read(Fraction& f) { cin >> f.n_ >> f.d_; f.Reduce(); }
```

- Unary Arithmetic Operations: Negate, Preserve (Sign), Componendo, and Dividendo

```
Fraction  Negate() const   { return Fraction(-n_, d_); }
Fraction  Preserve() const { return *this; }
Fraction& Componendo()     { return *this = Add(Fraction::UNITY); }
Fraction& Dividendo()      { return *this = Subtract(Fraction::UNITY); }
```

Implementation of Fraction: Version 1

- Binary Arithmetic Operations: Add, Subtract, Multiply, Divide, and Modulus

```
Fraction Add(const Fraction& f2) const {
    unsigned int d = Fraction::lcm(d_, f2.d_);
    int n = n_*(d / d_) + f2.n_*(d / f2.d_);
    return Fraction(n, d);
}
Fraction Subtract(const Fraction& f2) const { return Add(f2.Negate()); }
Fraction Multiply(const Fraction& f2) const {
    return Fraction(n_*f2.n_, d_*f2.d_);
}
Fraction Divide(const Fraction& f2) const { return Multiply(f2.Invert()); }
Fraction Modulus(const Fraction& f2) const {
    if (f2.n_ == 0) { throw "Divide by 0 is undefined\n"; }
    Fraction tf = Divide(f2);
    Fraction f = Subtract(Fraction(static_cast<int>(tf.n_ / tf.d_)).Multiply(f2));

    return f;
}
```

Implementation of Fraction: Version 1

- Binary Relational Operations: Less, LessEq, More, MoreEq, Eq, NotEq

```
bool Eq(const Fraction& f2) const     { return ((n_ == f2.n_) && (d_ == f2.d_)); }
bool NotEq(const Fraction& f2) const  { return !(Eq(f2)); }
bool Less(const Fraction& f2) const   { return Subtract(f2).n_ < 0; }
bool LessEq(const Fraction& f2) const { return !More(f2); }
bool More(const Fraction& f2) const   { return Subtract(f2).n_ > 0; }
bool MoreEq(const Fraction& f2) const { return !Less(f2); }
```

- Extended Operations: Invert and Convert to double

```
Fraction Invert() const { // Inverts a fraction. !(p/q) = q/p
    if (d_ == 0)  throw "Divide by 0 is undefined\n";
    return Fraction(d_, n_);
}
double Double() const { // Converts to a double
    return static_cast<double>(n_) / static_cast<double>(d_);
}
```

- Static constant fractions

```
static const Fraction UNITY; // Defines 1/1
static const Fraction ZERO;  // Defines 0/1
```

Implementation of Fraction: Version 1

- Support Functions: gcd, lcm and reduce: Should be `private` - not part of interface

```
static int gcd(int a, int b) { // Finds the gcd for two +ve integers
    while (a != b)
        if (a > b) a = a - b;
        else b = b - a;
    return a;
}
static int lcm(int a, int b) { // Finds the lcm for two +ve integers
    return (a / gcd(a, b))*b;
}
Fraction& Reduce() { // Reduces a fraction
    if (d_ == 0) { throw "Fraction with Denominator 0 is undefined"; }
    if (d_ < 0) { n_ = -n_;
        d_ = static_cast<unsigned int>(-static_cast<int>(d_));
        return *this;
    }
    if (n_ == 0) { d_ = 1; return *this; }
    unsigned int n = (n_ > 0) ? n_ : -n_, g = gcd(n, d_);
    n_ /= static_cast<int>(g); // as n_ is int and g is unsigned int the division may not work
    d_ /= g;
    return *this;
}
```

Testing Fraction: Version 1: Test Application


```
#include <iostream>
using namespace std;
#include "Fraction.h"

int main() {
    cout << "Construction, Copy Operations and Write Test" << endl; // Ctor, Copy & Write Test
    Fraction f1(5, 3); cout << "Fraction f1(5, 3) = "; Fraction::Write(f1); cout << endl;
    Fraction f2(7);    cout << "Fraction f2(7) = "; Fraction::Write(f2); cout << endl;
    Fraction f3;       cout << "Fraction f3 = "; Fraction::Write(f3); cout << endl;
    Fraction f4(f1);   cout << "Fraction f4(f1) = "; Fraction::Write(f4); cout << endl;
    Fraction f5(3, 6); cout << "Fraction f5(3, 6) = "; Fraction::Write(f5); cout << endl;
    Fraction f6(0, 4); cout << "Fraction f6(0, 4) = "; Fraction::Write(f6); cout << endl;
    cout << "Assignment: f2 = f1: f2 = "; Fraction::Write(f2 = f1); cout << endl << endl;

    cout << "Read Test" << endl; // Read Test
    cout << "Read f1 = "; Fraction::Read(f1); Fraction::Write(f1); cout << endl << endl;

    f1 = Fraction(2, 5); /* Using f1 for the following tests */ f2 = f1; // Copy to restore f1 later
    cout << "Unary Ops Test: Using f1 = ";
    Fraction::Write(f1); cout << " for all" << endl; // Unary Operations Test
    cout << "Negate: f1.Negate() = "; Fraction::Write(f1.Negate()); cout << endl;
    cout << "Preserve: f1.Preserve() = "; Fraction::Write(f1.Preserve()); cout << endl;
    cout << "Componendo: f1.Componendo() = "; Fraction::Write(f1.Componendo()); cout << endl; f1 = f2;
    cout << "Dividendo: f1.Dividendo() = "; Fraction::Write(f1.Dividendo()); cout << endl << endl;
```

Testing Fraction: Version 1: Test Application


```
f1 = Fraction(5, 12); f2 = Fraction(7, 18); // Using f1 and f2 for the following test
cout << "Binary Ops Test: Using f1 = "; // Binary Operations Test
Fraction::Write(f1); cout << ". f2 = "; Fraction::Write(f2); cout << " for all" << endl;
cout << "Binary Plus: f1.Add(f2) = "; Fraction::Write(f1.Add(f2));  cout << endl;
cout << "Binary Minus: f1.Subtract(f2) = "; Fraction::Write(f1.Subtract(f2));  cout << endl;
cout << "Binary Multiply: f1.Multiply(f2) = "; Fraction::Write(f1.Multiply(f2));  cout << endl;
cout << "Binary Divide: f1.Divide(f2) = "; Fraction::Write(f1.Divide(f2));  cout << endl;
cout << "Binary Residue: f1.Modulus(f2) = "; Fraction::Write(f1.Modulus(f2));  cout << endl << endl;

// Using f1 = Fraction(5, 12); f2 = Fraction(7, 18); for the following tests
cout << "Logical Ops Test: Using f1 = "; // Logical Operations Test
Fraction::Write(f1); cout << ". f2 = "; Fraction::Write(f2); cout << " for all" << endl;
cout << "Equal: " << ((f1.Eq(f2)) ? "true" : "false") << endl;
cout << "Not Equal: " << ((f1.NotEq(f2)) ? "true" : "false") << endl;
cout << "Less: " << ((f1.Less(f2)) ? "true" : "false") << endl;
cout << "Less Equal: " << ((f1.LessEq(f2)) ? "true" : "false") << endl;
cout << "Greater: " << ((f1.More(f2)) ? "true" : "false") << endl;
cout << "Greater Equal: " << ((f1.MoreEq(f2)) ? "true" : "false") << endl << endl;

// Using f1 = Fraction(5, 12); for the following tests
cout << "Extended Ops Test: Using f1 = "; // Extended Operations Test
Fraction::Write(f1); cout << " for all" << endl;
cout << "Invert: f1.Invert() = "; Fraction::Write(f1.Invert()); cout << endl;
cout << "Double: f1.Double() = "; cout << f1.Double() << endl << endl;
```

Testing Fraction: Version 1: Test Application


```
    cout << "Static Constants Test" << endl; // Static Constants Test
    cout << "UNITY = "; Fraction::Write(Fraction::UNITY); cout << endl;
    cout << "ZERO = "; Fraction::Write(Fraction::ZERO); cout << endl << endl;
}
```

```
Construction, Copy Operations and Write Test
Fraction f1(5, 3) = 5/3
Fraction f2(7) = 7
Fraction f3 = 1
Fraction f4(f1) = 5/3
Fraction f5(3, 6) = 1/2
Fraction f6(0, 4) = 0
Assignment: f2 = f1: f2 = 5/3

Read Test
2 7
Read f1 = 2/7

Unary Ops Test: Using f1 = 2/5
Negate: f1.Negate() = -2/5
Preserve: f1.Preserve() = 2/5
Componendo: f1.Componendo() = 7/5
Dividendo: f1.Dividendo() = -3/5

All tests passed
```

```
Binary Ops Test: Using f1 = 5/12. f2 = 7/18 for all
Binary Plus: f1.Add(f2) = 29/36
Binary Minus: f1.Subtract(f2) = 1/36
Binary Multiply: f1.Multiply(f2) = 35/216
Binary Divide: f1.Divide(f2) = 15/14
Binary Residue: f1.Modulus(f2) = 1/36

Logical Ops Test: Using f1 = 5/12. f2 = 7/18 for all
Equal: false
Not Equal: true
Less: false
Less Equal: false
Greater: true
Greater Equal: true

Extended Ops Test: Using f1 = 5/12 for all
Invert: f1.Invert() = 12/5
Double: f1.Double() = 0.416667

Static Constants Test
UNITY = 1
ZERO = 0
```

Fraction: Version 1

- So now we have one design and implementation for `Fraction` objects that can be manipulated by various operation member functions
- However, it still leaves a lot more to be desired. Consider, that we want to evaluate the following fraction expression:

$$f1 = \frac{2}{3}$$
$$f2 = \frac{8}{1}$$
$$f3 = \frac{5}{6}$$
$$f4 = (f1 + f2)/(f1 - f2)+!f3 - f2 * f3 = -\frac{1097}{165}$$

- Using Version 1:

```
void MixedText() { Fraction f1(2, 3), f2(8), f3(5, 6), f4;

    f4 = f1.Add(f2).Divide(f1.Subtract(f2)).Add(f3.Invert()).Subtract(f2.Multiply(f3));
    Fraction::Write(f4); cout << endl;
}
```

- *Horrendously complicated and error-prone, to say the least*
- To simplify, we map the member functions to various overloaded operators in Version 2

Design of Fraction: Interface: Version 2

- Construction, Destruction, and Copy Operations

```
explicit Fraction(int = 1, int = 1);  // Three overloads including a default constructor
~Fraction();                          // No virtual destructor needed
Fraction(const Fraction&);            // Copy constructor
Fraction& operator=(const Fraction&); // Copy assignment operator
```

- IO Operations: Read and Write (`friend` function needed for `iostream` support)

```
friend ostream& operator<<(ostream&, const Fraction&); // Write()
friend istream& operator>>(istream&, Fraction&);       // Read()
```

- Unary Arithmetic Operations: Preserve (Sign), Negate, Componendo, and Dividendo. Postfix operators are additions here

```
Fraction  operator+() const; // Preserve()
Fraction  operator-() const; // Negate()
Fraction& operator++();      // Pre-increment. Componendo(): p/q <-- p/q + 1
Fraction& operator--();      // Pre-decrement. Dividendo(): p/q <-- p/q - 1
Fraction  operator++(int);   // Post-increment.
                             // Lazy Componendo. p/q <-- p/q + 1. Returns old p/q
Fraction  operator--(int);   // Post-decrement.
                             // Lazy Dividendo. p/q <-- p/q - 1. Returns old p/q
```

Design of Fraction: Interface: Version 2

- Binary Arithmetic Operations: Add, Subtract, Multiply, Divide, and Modulus

```
Fraction operator+(const Fraction&) const; // Add()
Fraction operator-(const Fraction&) const; // Subtract()
Fraction operator*(const Fraction&) const; // Multiply()
Fraction operator/(const Fraction&) const; // Divide()
Fraction operator%(const Fraction&) const; // Modulus()
```

Since the constructor of `Fraction` is `explicit`, an `int` cannot be implicitly converted to `Fraction`. So we do not expect an addition operation like `i + f` where `int i;` and `Fraction`. Hence, member function operators are okay. Otherwise, we will need `friend` function operators:

```
friend Fraction operator+(const Fraction&, const Fraction&);
friend Fraction operator-(const Fraction&, const Fraction&);
friend Fraction operator*(const Fraction&, const Fraction&);
friend Fraction operator/(const Fraction&, const Fraction&);
friend Fraction operator%(const Fraction&, const Fraction&);
```

Design of Fraction: Interface: Version 2

- Advanced Assignment Operators. These are additions here:

```
Fraction& operator+=(const Fraction&);
Fraction& operator-=(const Fraction&);
Fraction& operator*=(const Fraction&);
Fraction& operator/=(const Fraction&);
Fraction& operator%=(const Fraction&);
```

- Binary Relational Operations: Less, LessEq, More, MoreEq, Eq, NotEq

```
bool operator==(const Fraction&) const; // Eq()
bool operator!=(const Fraction&) const; // NotEq()
bool operator<(const Fraction&) const;  // Less()
bool operator<=(const Fraction&) const; // LessEq()
bool operator>(const Fraction&) const;  // More()
bool operator>=(const Fraction&) const; // MoreEq()
```

Design of Fraction: Interface: Version 2

- Extended Operations: Invert and Convert to `double`

```
Fraction operator!() const; // Invert()
operator double();          // Double()
```

- Static constant fractions

```
static const Fraction UNITY; // Defines 1/1
static const Fraction ZERO;  // Defines 0/1
```

- Support Functions: gcd, lcm and reduce: Should be `private` - not part of interface

```
static int gcd(int, int); // Finds the gcd for two +ve integers
static int lcm(int, int); // Finds the lcm for two +ve integers
Fraction& operator*();    // Reduce()
```

  Since reduction is not on the interface, we may not overload an operator for it - it will be fine to use the earlier `Reduce()` function

- *Note that Bit-wise operators, Shift operators etc. are not overloaded in* `Fraction` *since there is no semantic interpretation for them*

Implementation of Fraction: Version 2

- Construction, Destruction, and Copy Operations

```
explicit Fraction(int n = 1, int d = 1): // Three overloads
    n_(d < 0 ? -n : n), d_(d < 0 ? -d : d) // d_ is unsigned int. So no -ve value
{ *(*this); } // Reduces the fraction by operator*()
Fraction(const Fraction& f) : n_(f.n_), d_(f.d_) { } // Copy Constructor
~Fraction() { } // No virtual destructor needed
Fraction& operator=(const Fraction& f) { n_ = f.n_; d_ = f.d_; return *this; }
```

- IO Operations: Read and Write (`friend` function needed for `iostream` support)

```
friend ostream& operator<<(ostream& os, const Fraction& f) { os << f.n_;
    if ((f.n_ != 0) && (f.d_ != 1)) os << "/" << f.d_; // Suppress denominator
    return os;                                         // if n_ == 0 or d_ == 1
}
friend istream& operator>>(istream& is, Fraction& f) { is >> f.n_ >> f.d_;
    *f; /* Reduces the fraction by operator*() */ return is;
}
```

- Unary Arithmetic Operations: Preserve, Negate, Componendo, Dividendo, & Postfix operators:

```
Fraction operator-() const { return Fraction(-n_, d_); }
Fraction operator+() const { return *this; }
Fraction& operator++()     { return *this += Fraction::UNITY; }
Fraction& operator--()     { return *this -= Fraction::UNITY; }
Fraction operator++(int)   { Fraction f = *this; ++*this; return f; }
Fraction operator--(int)   { Fraction f = *this; --*this; return f; }
```

Implementation of Fraction: Version 2

- Binary Arithmetic Operations: Add, Subtract, Multiply, Divide, and Modulo:

```
Fraction operator+(const Fraction& f2) const {
    unsigned int d = lcm(d_, f2.d_);
    int n = n_*(d / d_) + f2.n_*(d / f2.d_);
    return Fraction(n, d);
}
Fraction operator-(const Fraction& f2) const { return *this + (-f2); }
Fraction operator*(const Fraction& f2) const { return Fraction(n_*f2.n_, d_*f2.d_); }
Fraction operator/(const Fraction& f2) const {
    if (f2.n_ == 0) { throw "Divide by 0 is undefined\n"; }
    return Fraction(n_*f2.d_, d_*f2.n_);
}
Fraction operator%(const Fraction& f2) const {
    if (f2.n_ == 0) { throw "Divide by 0 is undefined\n"; }
    Fraction tf = (*this) / f2;
    return (*this) - Fraction(tf - Fraction(tf.n_ % tf.d_, tf.d_))*f2;
    // return (*this) - Fraction(static_cast<int>(tf.n_ / tf.d_))*f2; // As in Ver 1
}
```

Implementation of Fraction: Version 2

- Advanced Assignment Operators. These are additions here:

```
Fraction& operator+=(const Fraction& f) { *this = *this + f; return *this; }
Fraction& operator-=(const Fraction& f) { *this = *this - f; return *this; }
Fraction& operator*=(const Fraction& f) { *this = *this * f; return *this; }
Fraction& operator/=(const Fraction& f) { *this = *this / f; return *this; }
Fraction& operator%=(const Fraction& f) { *this = *this % f; return *this; }
```

- Binary Relational Operations: Less, LessEq, More, MoreEq, Eq, NotEq

```
bool operator==(const Fraction& f2) const { return ((n_ == f2.n_) && (d_ == f2.d_)); }
bool operator!=(const Fraction& f2) const { return !(*this == f2); }
bool operator<(const Fraction& f2) const  { return (*this - f2).n_ < 0; }
bool operator<=(const Fraction& f2) const { return !(*this > f2); }
bool operator>(const Fraction& f2) const  { return (*this - f2).n_ > 0; }
bool operator>=(const Fraction& f2) const { return !(*this < f2); }
```

Implementation of Fraction: Version 2

- Extended Operations: Invert and Convert to `double`

```
Fraction operator!() const { // Inverts a fraction. !(p/q) = q/p
    if (d_ == 0) { throw "Divide by 0 is undefined\n"; }
    return Fraction(d_, n_);
}
operator double() const { return static_cast<double>(n_) / static_cast<double>(d_); }
```

- Static constant fractions

```
static const Fraction UNITY; // Defines 1/1
static const Fraction ZERO;  // Defines 0/1s
```

- Support Functions: gcd, lcm and reduce: Should be `private` - not part of interface

```
static int gcd(int a, int b);
static int lcm(int a, int b);

Fraction& operator*() { // Reduces a fraction
    if (d_ == 0) { throw "Fraction with Denominator 0 is undefined"; }
    if (d_ < 0) { n_ = -n_; d_ = static_cast<unsigned int>(-static_cast<int>(d_)); return *this; }
    if (n_ == 0) { d_ = 1; return *this; }
    unsigned int n = (n_ > 0) ? n_ : -n_, g = gcd(n, d_);
    n_ /= static_cast<int>(g); // as n_ is int and g is unsigned int the division may not work
    d_ /= g;
    return *this;
}
```

Testing Fraction: Version 2: Pass Test Application


```
#include <iostream>
using namespace std;
#include "Fraction.h"
int main() {
    cout << "Construction, Copy Operations and Write Test" << endl; // Ctor, Copy & and Write Test
    Fraction f1(5, 3); cout << "Fraction f1(5, 3) = " << f1 << endl;
    Fraction f2(7);    cout << "Fraction f2(7) = " << f2 << endl;
    Fraction f3;       cout << "Fraction f3 = " << f3 << endl;
    Fraction f4(f1);   cout << "Fraction f4(f1) = " << f4 << endl;
    Fraction f5(3, 6); cout << "Fraction f5(3, 6) = " << f5 << endl;
    Fraction f6(0, 4); cout << "Fraction f6(0, 4) = " << f6 << endl;
    cout << "Assignment: f2 = f1: f2 = " << (f2 = f1) << endl << endl;

    cout << "Read Test" << endl; // Read Test
    cin >> f1; cout << "Read f1 = " << f1 << endl << endl;

    f1 = Fraction(2, 5); /* Using f1 for the following tests */ f2 = f1; // Copy to restore f1 later
    cout << "Unary Ops Test: Using f1 = " << f1 << " for all" << endl; // Unary Operations Test
    cout << "Negate: -f1 = " << -f1 << endl;
    cout << "Preserve: +f1 = " << +f1 << endl;
    cout << "Componendo: ++f1 = " << ++f1 << endl; f1 = f2;
    cout << "Dividendo: --f1 = " << --f1 << endl; f1 = f2;
    cout << "Lazy Componendo: f1++ = " << f1++ << " Lazy f1 = " << f1 << endl; f1 = f2;
    cout << "Lazy Dividendo: f1-- = " << f1-- << " Lazy f1 = " << f1 << endl << endl;
```

Testing Fraction: Version 2: Pass Test Application


```
f1 = Fraction(5, 12); f2 = Fraction(7, 18); // Using f1 and f2 for the following test

// Binary Operations Test
cout << "Binary Ops Test: Using f1 = " << f1 << ". f2 = " << f2 << " for all" << endl;
cout << "Binary Plus: f1 + f2 = " << (f1 + f2) << endl;
cout << "Binary Minus: f1 - f2 = " << (f1 - f2) << endl;
cout << "Binary Multiply: f1 * f2 = " << (f1 * f2) << endl;
cout << "Binary Divide: f1 / f2 = " << (f1 / f2) << endl;
cout << "Binary Residue: f1 % f2 = " << (f1 % f2) << endl << endl;

// Using f1 = Fraction(5, 12); f2 = Fraction(7, 18); for the following tests
f3 = f1; // Copy to restore f1 later
// Binary Assignment Operations Test
cout << "Binary Assignment Ops Test: Using f1 = " << f1 << ". f2 = " << f2 << " for all" << endl;
cout << "Plus Assign: f1 += f2: f1 = " << (f1 += f2) << endl; f1 = f3;
cout << "Minus Assign: f1 -= f2: f1 = " << (f1 -= f2) << endl; f1 = f3;
cout << "Multiply Assign: f1 *= f2: f1 = " << (f1 *= f2) << endl; f1 = f3;
cout << "Divide Assign: f1 /= f2: f1 = " << (f1 /= f2) << endl; f1 = f3;
cout << "Residue Assign: f1 %= f2: f1 = " << (f1 %= f2) << endl << endl; f1 = f3;
```

Testing Fraction: Version 2: Pass Test Application


```
    // Using f1 = Fraction(5, 12); f2 = Fraction(7, 18); for the following tests
    // Logical Operations Test
    cout << "Logical Ops Test: Using f1 = " << f1 << ". f2 = " << f2 << " for all" << endl;
    cout << "Equal: " << ((f1 == f2) ? "true" : "false") << endl;
    cout << "Not Equal: " << ((f1 != f2) ? "true" : "false") << endl;
    cout << "Less: " << ((f1 < f2) ? "true" : "false") << endl;
    cout << "Less Equal: " << ((f1 <= f2) ? "true" : "false") << endl;
    cout << "Greater: " << ((f1 > f2) ? "true" : "false") << endl;
    cout << "Greater Equal: " << ((f1 >= f2) ? "true" : "false") << endl << endl;

    // Extended Operations Test
    // Using f1 = Fraction(5, 12); for the following tests
    cout << "Extended Ops Test: Using f1 = " << f1 << " for all" << endl;
    cout << "Invert: !f1 = " << !f1 << endl;
    cout << "Double: (double)f1 = "; cout << static_cast<double>(f1) << endl << endl;

    // Static Constants Test
    cout << "Static Constants Test" << endl;
    cout << "UNITY = " << Fraction::UNITY << endl;
    cout << "ZERO = " << Fraction::ZERO << endl << endl;
}
```

Testing Fraction: Version 2: Pass Test Application


```
Construction, Copy Operations and Write Test
Fraction f1(5, 3) = 5/3
Fraction f2(7) = 7
Fraction f3 = 1
Fraction f4(f1) = 5/3
Fraction f5(3, 6) = 1/2
Fraction f6(0, 4) = 0
Assignment: f2 = f1: f2 = 5/3

Read Test
2 7
Read f1 = 2/7

Unary Ops Test: Using f1 = 2/5 for all
Negate: -f1 = -2/5
Preserve: +f1 = 2/5
Componendo: ++f1 = 7/5
Dividendo: --f1 = -3/5
Lazy Componendo: f1++ = 2/5 Lazy f1 = 7/5
Lazy Dividendo: f1-- = 2/5 Lazy f1 = -3/5

Binary Ops Test: Using f1 = 5/12. f2 = 7/18
Binary Plus: f1 + f2 = 29/36
Binary Minus: f1 - f2 = 1/36
All tests passed
```

```
Binary Multiply: f1 * f2 = 35/216
Binary Divide: f1 / f2 = 15/14
Binary Residue: f1 % f2 = 1/36

Binary Assignment Ops Test: Using f1 = 5/12. f2 = 7/18
Plus Assign: f1 += f2: f1 = 29/36
Minus Assign: f1 -= f2: f1 = 1/36
Multiply Assign: f1 *= f2: f1 = 35/216
Divide Assign: f1 /= f2: f1 = 15/14
Residue Assign: f1 %= f2: f1 = 1/36

Logical Ops Test: Using f1 = 5/12. f2 = 7/18 for all
Equal: false
Not Equal: true
Less: false
Less Equal: false
Greater: true
Greater Equal: true

Extended Ops Test: Using f1 = 5/12 for all
Invert: !f1 = 12/5
Double: (double)f1 = 0.416667

Static Constants Test
UNITY = 1
ZERO = 0
```

Testing Fraction: Version 2: Fail Test Application


```
int main() {
    try { cout << "Construct Fraction (1, 0): ";
        Fraction f1(1, 0); // Construct Fraction (1, 0): Fraction with Denominator 0 is undefined
    } catch (const char* s) { cout << s << endl; } cout << endl;
    Fraction f1;
    try { cout << "Read f1 = "; // Read f1 = 1 0
        cin >> f1; cout << f1 << endl; // Fraction with Denominator 0 is undefined
    } catch (const char* s) { cout << s << endl; } cout << endl;
    f1 = Fraction(5, 12); Fraction f2 = Fraction::ZERO, f3;
    try { cout << "Binary Divide: f3 = " << f1 << " / " << f2 << ": ";
        f3 = f1 / f2; cout << f3 << endl; // Binary Divide: f3 = 5/12 / 0: Divide by 0 is undefined
    } catch (const char* s) { cout << s << endl; }
    try { cout << "Binary Residue: f3 = " << f1 << " % " << f2 << ": ";
        f3 = f1 % f2; cout << f3 << endl; // Binary Residue: f3 = 5/12 % 0: Divide by 0 is undefined
    } catch (const char* s) { cout << s << endl; }
    try { cout << "Divide Assign: f1 = " << f1 << " /= " << f2 << ": ";
        f1 /= f2; cout << f1 << endl; // Divide Assign: f1 = 5/12 /= 0: Divide by 0 is undefined
    } catch (const char* s) { cout << s << endl; }
    try { cout << "Residue Assign: f1 = " << f1 << " %= " << f2 << ": ";
        f1 %= f2; cout << f1 << endl; // Residue Assign: f1 = 5/12 %= 0: Divide by 0 is undefined
    } catch (const char* s) { cout << s << endl; }
    try { cout << "Invert: f1 = " << " ! " << f2 << ": ";
        f1 = !f2; cout << f1 << endl; // Invert: f1 =  ! 0: Fraction with Denominator 0 is undefined
    } catch (const char* s) { cout << s << endl; }
}
```

Cross Version Test

- To assess Verion 2 against Version 1, again consider the following fraction expression:

$$\begin{aligned} f1 &= \frac{2}{3} \\ f2 &= \frac{8}{1} \\ f3 &= \frac{5}{6} \\ f4 &= (f1 + f2)/(f1 - f2)+!f3 - f2 * f3 = -\frac{1097}{165} \end{aligned}$$

- *Using* **Version 1**: *Very easy to get confused in the chain of calls and parentheses*

```
void MixedText() { Fraction f1(2, 3), f2(8), f3(5, 6), f4;

    f4 = f1.Add(f2).Divide(f1.Subtract(f2)).Add(f3.Invert()).Subtract(f2.Multiply(f3));
    Fraction::Write(f4); cout << endl;
}
```

- *Using* **Version 2**: *Just as we write the algebra*

```
void MixedText() { Fraction f1(2, 3), f2(8), f3(5, 6), f4;

    f4 = (f1 + f2) / (f1 - f2) +!f3 - f2 * f3;
    cout << f4 << endl;
}
```

Tutorial Summary

- Analysed the difference between Built-in & UDT
- Discussed the meaning of Building a data type
- Understood the necessity of Building a data type
- Built a Fraction data type by iterative refinement